



ITM Skills University

School of Future Tech

Case Study Report

on

**MicroVest — A Fractional Stock Investment Simulation
Platform**

by

Aaryan Kuchekar

150096725048

Academic Year 2025–2026

Index

1. Introduction to the Case Study	3
2. Problem Statement / Case Background (Abstract)	5
3. Case Study Design	7
3.1 Project Architecture	
3.2 File & Folder Structure	
3.3 System Architecture Diagram	
3.4 User Interaction Flow Diagram	
3.5 Design System	
4. HTML5 — Concepts, Tags & Usage in MicroVest	11
4.1 What is HTML5?	
4.2 Semantic HTML	
4.3 Document Structure Tags	
4.4 Semantic Landmark Tags	
4.5 Form & Input Elements	
4.6 Table Elements	
4.7 Media & Graphics — Inline SVG	
4.8 HTML Attributes (Standard & Custom data-*)	
4.9 ARIA Accessibility Attributes	
4.10 Complete HTML Tag Reference Table	
5. CSS3 — Concepts, Properties & Usage in MicroVest	19
5.1 What is CSS3?	
5.2 CSS Custom Properties (Variables)	
5.3 The Cascade and Specificity	
5.4 Box Model	
5.5 Flexbox	
5.6 CSS Grid	
5.7 Positioning	
5.8 CSS Animations & Transitions	
5.9 Glassmorphism & Visual Effects	
5.10 Responsive Design & Media Queries	
5.11 CSS Theming System (Dark/Light Mode)	
5.12 Complete CSS Properties Reference Table	
6. JavaScript — Concepts, Patterns & Usage in MicroVest	29
6.1 What is Vanilla JavaScript?	
6.2 Object-Oriented Programming — Classes & Private Fields	
6.3 The Observer Pattern — subscribe()	
6.4 DOM Selection & Manipulation	
6.5 Events & Event Delegation	
6.6 setInterval & the Price Simulation Loop	

6.7	ES6+ Features Used	
6.8	Data Structures — Map & Array	
6.9	localStorage API — State Persistence	
6.10	Mathematical Algorithms — GBM, Weighted Average, Gain/Loss	
6.11	SVG Chart Generation via JS	
6.12	Complete JavaScript Patterns Reference Table	
7.	Implementation Details & Snapshots	41
8.	Results and Conclusion	43
9.	References	45

1. Introduction to the Case Study

The Problem with Learning Investing

The study of stock market investing is a cornerstone of modern financial education. Students enrolled in commerce, finance, and technology programmes across India are routinely introduced to concepts such as equity shares, dividends, portfolio diversification, and risk management. However, a critical and often overlooked gap exists between this theoretical exposure and practical competence. Most students understand what a share of stock is in principle, but have never actually purchased one, observed live price movements, managed a portfolio, or experienced the psychological effect of watching an investment gain or lose value in real time. This gap is not a failure of curriculum design — it is a structural consequence of the financial barrier to entry. Real investing requires real capital, and every mistake carries real financial consequences. For a student or young professional without disposable savings, the cost of learning by doing is simply too high.

The problem is compounded in the Indian market context. Popular blue-chip stocks listed on the National Stock Exchange of India trade at prices that are inaccessible for beginners. Tata Consultancy Services (TCS) trades at approximately Rs. 3,940 per share, Reliance Industries at Rs. 2,850 per share, and HDFC Bank at approximately Rs. 1,760 per share. A student wishing to build a diversified portfolio across even five such companies would need to commit upwards of Rs. 15,000 to Rs. 20,000 — and that is before brokerage fees, account opening charges, and the knowledge required to avoid catastrophic early losses. The result is a generation of students who can describe the difference between a bull market and a bear market in a written examination, but who have never developed the market intuition that only comes from hands-on experience. MicroVest was built explicitly to close this gap.

The MicroVest Solution

MicroVest is a fully browser-based fractional investment simulation platform designed to give any student access to a realistic, risk-free investing experience. The core innovation is fractional investing: instead of requiring a user to purchase a whole share of TCS at Rs. 3,940, MicroVest allows them to invest as little as Rs. 10 and receive a proportional fraction of a share — in this case, approximately 0.00254 shares. The user receives exactly the same proportional gains or losses as a full shareholder. If TCS rises by 2%, a Rs. 10 investment grows to Rs. 10.20, just as a Rs. 3,940 investment would grow to Rs. 4,018.80. This makes it possible for any student, regardless of financial background, to build a genuinely diversified portfolio across all 10 simulated Indian stocks with a virtual budget of just Rs. 100.

The platform simulates real market conditions using a mathematical model called Geometric Brownian Motion (GBM) — the same stochastic model used by quantitative analysts in professional finance to model asset price behaviour. GBM

generates price movements that are random but statistically bounded: they can move up or down, the magnitude of movement is proportional to the stock's assigned volatility parameter, and extreme drift over long periods is constrained by a drift guard. These price movements update every 2 seconds across all 10 stocks simultaneously, giving the user a visceral sense of how prices fluctuate in a live market. Beyond the trading simulation, MicroVest provides a dedicated education module where users can learn investing fundamentals through concept cards, a 15-term glossary, annotated diagrams, and a 5-question multiple choice quiz — tying theory directly to the practice available one click away.

Why Pure HTML, CSS, and JavaScript

MicroVest was intentionally built with zero external frameworks, libraries, or dependencies. There is no React, no Vue.js, no Angular, no Bootstrap, no jQuery, no Chart.js, and no Webpack build pipeline. Every feature — from the live price simulation engine to the glassmorphism card UI, from the SVG sparkline charts to the dark/light theme toggle — was implemented from scratch using only the native capabilities of the browser: HTML5, CSS3, and Vanilla JavaScript. This architectural decision was not a constraint; it was a deliberate pedagogical choice. The purpose of MicroVest as an academic project is to demonstrate deep mastery of foundational web technologies, and that mastery can only be demonstrated when there are no abstractions hiding the implementation.

When a developer builds with React, the framework manages the Document Object Model (DOM) on their behalf through a Virtual DOM diffing algorithm. When they use Bootstrap, pre-built CSS classes handle layout and responsiveness. When they integrate Chart.js, a library handles SVG or Canvas rendering. In each case, the developer never needs to understand precisely how the browser renders a layout, how CSS transitions are triggered, or how to map data coordinates to SVG pixel positions. MicroVest inverts this — the developer must understand all of these things, because they are responsible for all of them. The result is a codebase where every line of code serves a documented, understood purpose: the CSS cascade is explicitly managed through 7 layered files, the JavaScript class hierarchy is formally designed using the Observer pattern, and accessibility is implemented manually through ARIA attributes rather than inherited from a component library.

Platform Overview

MicroVest consists of four HTML pages that work together as a unified application. 'index.html' serves as the landing page, featuring a hero section, a three-step explanation of fractional investing, live ticker previews of all 10 stocks, and featured stock cards with real-time price updates. 'stocks.html' is the main trading interface where users can browse all 10 Indian stocks, filter them by risk level (Low, Medium, High), sort them by various criteria, search by ticker or company name, view live sparkline price charts, and open an inline calculator to simulate a purchase. 'portfolio.html' shows the user's current holdings with summary statistics — total

value, total invested, gain/loss in rupees, and gain/loss as a percentage — along with a visual allocation bar chart and a scrollable holdings table. 'education.html' provides a structured learning experience covering fractional investing concepts, risk level explanations, a glossary of 15 investment terms presented in an animated accordion, a 5-question knowledge quiz, and a technical explanation of how the GBM simulation works. Across these four pages, the project comprises 4 HTML files, 7 CSS files, and 6 JavaScript files, all working together as a single cohesive client-side application with no backend server required.

2. Problem Statement / Case Background

Background

Fractional investing is a relatively recent financial innovation that has been popularised by consumer investing platforms such as Groww, Zerodha, and Robinhood. The core premise is straightforward: a single share of stock is treated as a divisible asset, and a platform allows users to purchase any fraction of that share proportional to the amount they invest. For example, if Tata Consultancy Services (TCS) is trading at Rs. 3,940 and a user has only Rs. 100 to invest, they can purchase $100 / 3,940 = 0.02538$ shares of TCS. They receive the same percentage return as any full shareholder: if TCS gains 3% in a session, their holding grows from Rs. 100 to Rs. 103. Fractional investing eliminates the capital barrier that historically prevented retail investors from owning portions of expensive, high-quality equities. MicroVest implements this model in a simulated environment where the same arithmetic applies, but all prices are algorithmically generated rather than sourced from a live exchange.

MicroVest simulates the fractional investing model using a JavaScript class named 'PriceSimulator' that applies the Geometric Brownian Motion mathematical model to generate price changes every two seconds. GBM is used in quantitative finance to model the behaviour of stock prices because it captures two key properties of real markets: randomness (prices can move in any direction at any moment) and proportionality (larger prices tend to have larger absolute price changes, but similar percentage changes). In MicroVest, each of the 10 simulated stocks is assigned an individual volatility parameter that controls how dramatically its price moves. HDFC Bank is assigned a volatility of 0.005, reflecting its status as a stable, large-cap stock — its prices move in small, gentle increments. Adani Enterprises is assigned a volatility of 0.024, reflecting its historical behaviour as a high-volatility stock — its prices swing more aggressively. This gives the simulation genuine educational value: students can observe how the same investment amount behaves very differently across stocks with different volatility profiles.

Beyond price simulation and fractional share trading, MicroVest provides a comprehensive educational module on 'education.html'. This module is structured into six distinct sections. The first explains the concept of fractional investing through three illustrative concept cards. The second describes the three risk levels used in MicroVest — Low, Medium, and High — and explains what each means in terms of expected volatility and potential return. The third uses an annotated diagram to teach users how to read a stock card, identifying each piece of information displayed: the ticker symbol, company name, sector badge, current price, percentage change, risk badge, sparkline chart, and the Invest Now button. The fourth section presents a 15-term glossary of essential investment vocabulary — including terms like portfolio, diversification, yield, P/E ratio, and market capitalisation — in an animated accordion

interface. The fifth section tests the user's knowledge with a 5-question multiple choice quiz that provides immediate feedback and a score screen. The sixth explains, in accessible language, how the GBM simulation works and what the volatility parameter means.

Abstract

MicroVest is a client-side web application that simulates fractional stock investing in the Indian equity market context. The platform implements a live price simulation engine using a dampened Geometric Brownian Motion (GBM) model executed via JavaScript's `setInterval` API, firing at 2,000-millisecond intervals across 10 Indian stocks, each with individually calibrated volatility parameters chosen to reflect the risk profile of their real-world NSE-listed counterparts. The system implements fractional share calculations with a 0.5% platform fee deducted from each purchase, weighted average portfolio cost basis tracking using browser `localStorage` persistence, real-time gain/loss computation using the `Array.reduce()` higher-order function, and SVG-based sparkline and trend charts constructed entirely without any external charting library. The user interface demonstrates CSS3 glassmorphism achieved through backdrop-filter and rgba transparency, CSS custom properties forming a complete design token system that powers an instantaneous dark/light theming mechanism, CSS Grid and Flexbox for fully responsive layout across five tested breakpoints, and CSS `@keyframes` animations for the aurora background, card entrance staggering, and price flash effects. The project demonstrates mastery of semantic HTML5 document structure with ARIA accessibility attributes including `aria-label`, `aria-expanded`, and `aria-hidden`, the Observer design pattern implemented through a subscribe/notify architecture in JavaScript, Object-Oriented Programming with ES2022 private class fields for true encapsulation, event-driven DOM manipulation, and the browser's native `localStorage` and JSON APIs for stateful persistence — all implemented in pure browser-native code with zero framework dependency, zero build tooling, and zero external runtime dependencies.

3. Case Study Design

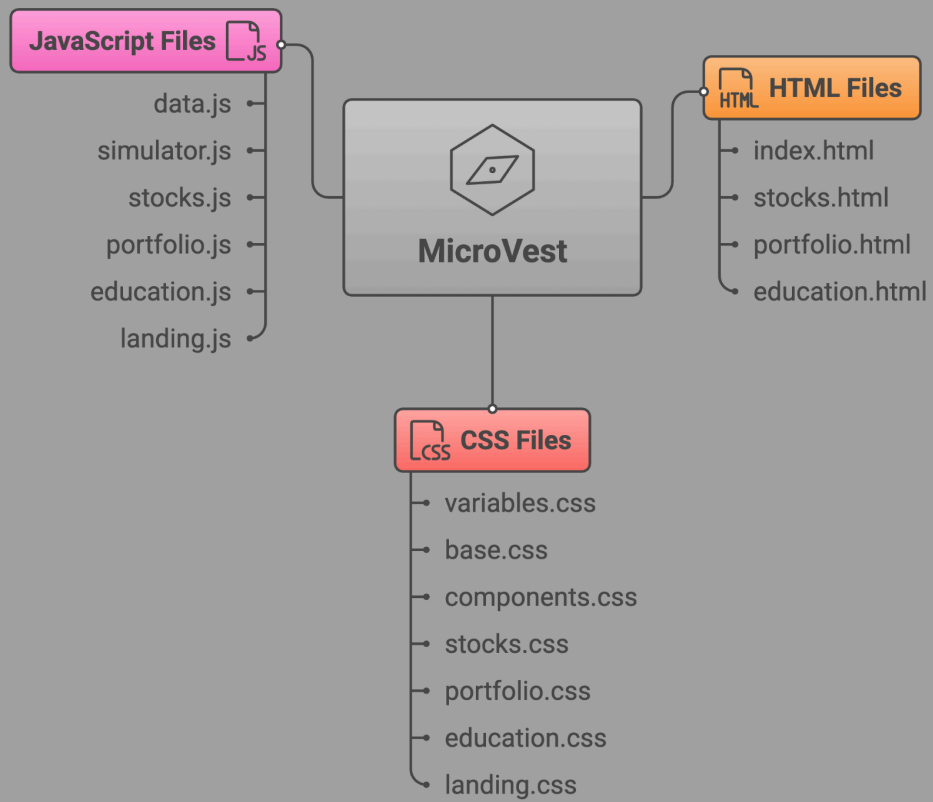
3.1 Project Architecture

MicroVest is designed around the software engineering principle of Separation of Concerns — the idea that each file should have exactly one clearly defined responsibility. The CSS layer is split into 7 files to enforce a strict cascade hierarchy. 'variables.css' is the single source of truth for every design token in the system — colours, font families, spacing units, border radii, transition durations, and glassmorphism values. All other CSS files reference these tokens using the CSS var() function, meaning no colour value is ever hardcoded outside of variables.css. 'base.css' handles the global reset, the aurora background animation, the sticky navbar, and all CSS @keyframes definitions. 'components.css' defines reusable UI components — the glass card mixin, button styles, badges, the accordion, and the buy calculator — that appear on multiple pages. Each of the four pages then has its own dedicated CSS file (stocks.css, portfolio.css, education.css, landing.css) that handles layout and styling specific to that page only.

The JavaScript layer follows the same discipline. 'data.js' contains only the STOCKS array — 10 objects each describing one Indian stock with its ticker, company name, sector, base price, and volatility — plus two constants: FEE_RATE (0.005 = 0.5%) and SIM_INTERVAL_MS (2000). 'simulator.js' contains only the PriceSimulator class and knows nothing about any HTML page. It is imported by both stocks.html and portfolio.html, and it works identically on both — its only job is to compute new prices using GBM, maintain price history, and notify all registered subscribers. Each page then has its own JavaScript file: 'stocks.js' handles filtering, sorting, searching, the calculator, the buy transaction, and sparkline updates; 'portfolio.js' loads holdings from localStorage, computes portfolio statistics, and renders the holdings table and trend chart; 'education.js' manages the accordion state machine and the quiz state machine; and 'landing.js' drives the landing page ticker preview, the theme toggle, and the navigation scroll effect. This architecture means any file can be read, debugged, or modified without needing to understand the entire codebase.

3.2 File & Folder Structure

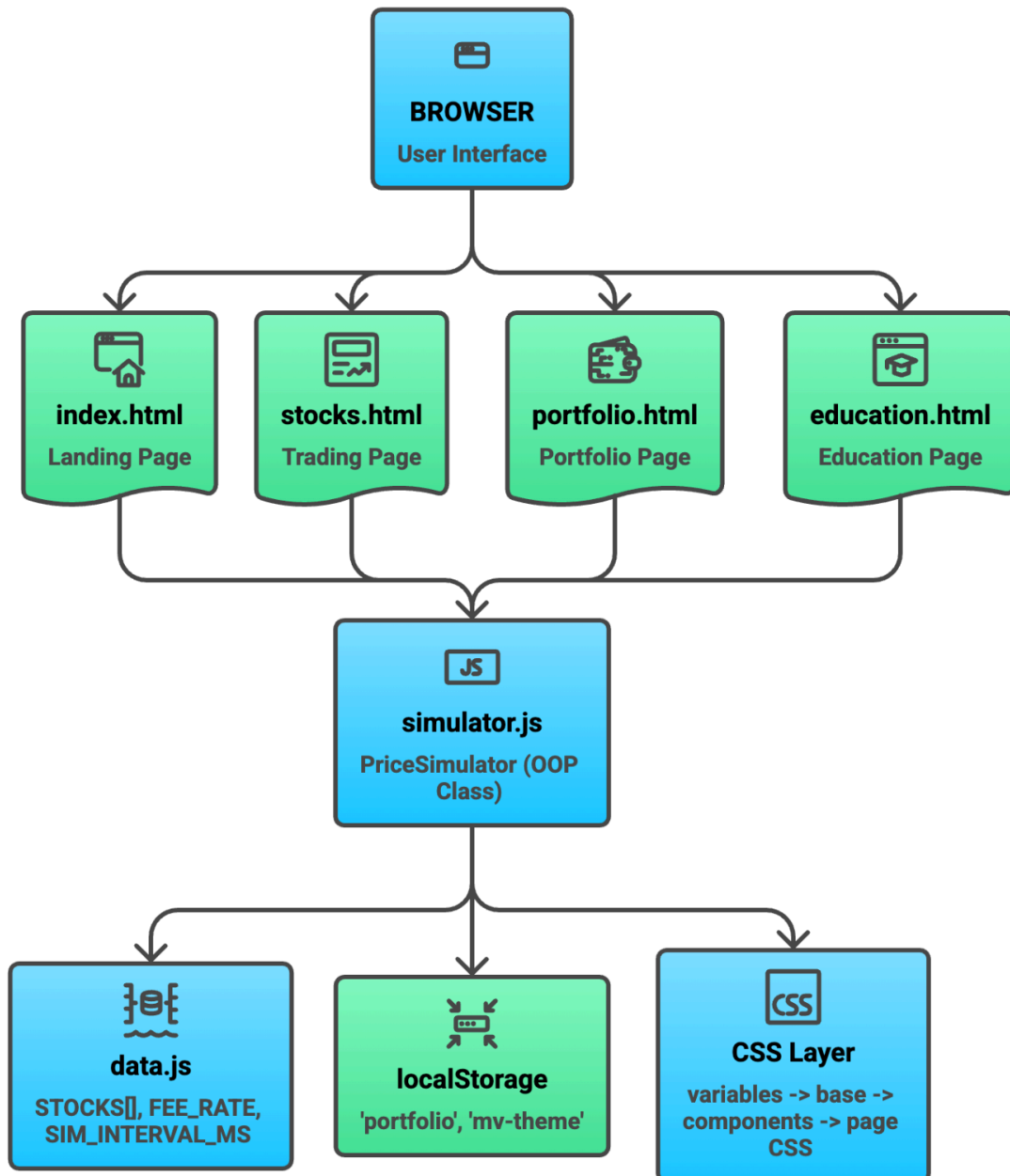
MicroVest Project Structure



Made with  Napkin

3.3 System Architecture Diagram

MicroVest System Architecture



Made with  Napkin

3.4 User Interaction Flow Diagram

User Journey Through MicroVest



Made with Napkin

3.5 Design System

MicroVest's visual identity is built around what was named the 'Deep Space Finance' design language — a deliberate aesthetic choice that positions the platform as a sophisticated, premium tool rather than a toy simulator. The foundational colour is a deep, near-black navy ('--bg-base: #080C14') that evokes the depth and seriousness of financial data dashboards. Against this background, two animated aurora radial gradients pulse in cyan (#00D4FF) and violet (#7B61FF) — these are CSS radial-gradient backgrounds on a fixed-position overlay that slowly drifts and scales via a @keyframes animation, creating a living, atmospheric backdrop that is visible through the semi-transparent glassmorphism cards. Stock price increases are universally coded green (#00E676) and decreases red (#FF3D57), inheriting the universal financial colour convention that users will recognise intuitively.

The typography system uses three typefaces, each chosen for a specific semantic purpose. Syne, a geometric display font, is used for all page headings and the site logo — its bold, structured letterforms communicate authority and clarity. DM Sans is used for all body text and UI labels — it is a neutral, highly legible humanist

sans-serif designed for screen reading at small sizes. Space Mono, a monospaced typeface, is used exclusively for all price values and number displays. This is not merely aesthetic: monospaced fonts ensure that every digit (0 through 9) occupies exactly the same width. When a stock price changes from Rs. 1,760.45 to Rs. 1,761.20 during a live update, the surrounding layout does not shift, because all digits have equal width. Using a proportional font for prices would cause subtle but visually jarring layout shifts every time a digit changed.

Token Name	Value	Usage
--bg-base	#080C14	Page background
--accent-cyan	#00D4FF	Primary brand, card borders, links
--accent-violet	#7B61FF	Secondary brand, aurora gradient
--accent-gold	#FFB830	Warnings, fee indicators
--up-strong	#00E676	Price increases, positive gain
--down-strong	#FF3D57	Price decreases, negative gain
--text-primary	#E8EFF8	Main text (dark mode)
--text-secondary	#8FA3BE	Subtitles, labels
--glass-bg	rgba(15,22,33,0.75)	Card backgrounds

4. HTML5 — Concepts, Tags & Usage in MicroVest

4.1 What is HTML5?

HyperText Markup Language (HTML) is the standard language used to create and structure content on the World Wide Web. It provides the skeleton of every web page: it defines what content exists on a page, what type that content is (a heading, a paragraph, a navigation menu, a data table, an image), and how content is organised into a hierarchical structure called the Document Object Model. HTML is not a programming language — it does not contain logic, conditionals, or loops. It is a markup language: every piece of content is wrapped in tags that tell the browser what the content means and how to handle it. HTML works alongside CSS (which controls how content looks) and JavaScript (which controls how content behaves), but HTML itself is solely responsible for structure and meaning.

HTML5, the fifth and current major revision of HTML, was standardised by the W3C and WHATWG in 2014 and has evolved continuously since. The most significant advancement of HTML5 over its predecessor HTML4 was the introduction of semantic elements — tags that carry inherent meaning about the role of their content. Where HTML4 required developers to use `<div id='header'>` and `<div id='nav'>` — generic containers with no intrinsic meaning — HTML5 provides `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, and `<footer>`. These elements tell the browser, search engines, screen readers, and other automated tools precisely what structural role each block of content plays. HTML5 also introduced the `<canvas>` element for pixel-level 2D drawing, native `<audio>` and `<video>` embedding without plugins, form input types such as `type='email'`, `type='number'`, and `type='search'` with built-in client-side validation, the `data-*` custom attribute system for embedding metadata in HTML elements, the `localStorage` and `sessionStorage` browser storage APIs (accessible via JavaScript), and formal ARIA integration for accessibility. MicroVest leverages all of these HTML5 capabilities across its four pages.

Every MicroVest page begins with the declaration `<!DOCTYPE html>`. This is not an HTML tag — it is a processing instruction to the browser's rendering engine. Without it, browsers enter 'quirks mode' — a backward-compatibility mode that emulates the broken CSS box model of Internet Explorer 5 and other legacy browsers. Quirks mode can cause margins, widths, and padding to be calculated differently from standards, breaking layouts that rely on box-sizing: border-box. With `<!DOCTYPE html>`, the browser renders in full standards mode, guaranteeing consistent, predictable rendering across all modern browsers. This single line is the minimum required declaration to identify a document as HTML5.

4.2 Semantic HTML — Why It Matters

Semantic HTML is the practice of using HTML elements that accurately describe the meaning and role of their content, rather than using generic containers styled with CSS. The distinction between semantic and non-semantic HTML is fundamental: a `<div>` element is a generic block-level container with no intrinsic meaning. A browser, a screen reader, or a search engine crawler that encounters a `<div>` knows only that it is a box — it learns nothing about what is inside. A `<header>` element, by contrast, communicates that this content is the introductory or navigational section of a page or region. A `<nav>` communicates that this content is a collection of navigation links. A `<main>` element communicates that this is the primary, unique content of the document. This semantic information is embedded in the HTML specification and is understood by every browser, search engine, and assistive technology that processes it.

The practical benefits of semantic HTML are substantial and span three dimensions: search engine optimisation, accessibility, and developer experience. From an SEO perspective, search engines such as Google assign higher relevance scores to content that is correctly semantically tagged. A page title inside a `<h1>` element is treated as the definitive topic of the page. Body content inside `<main>` is treated as primary content. Navigation links inside `<nav>` are recognised as site structure. A page built entirely with `<div>` tags provides no such signals, and its content may be ranked lower or misclassified. From an accessibility perspective, screen reader software used by visually impaired users depends entirely on semantic elements to create a navigable document outline. When a page contains a proper `<nav>`, `<main>`, and `<footer>`, a screen reader user can jump between these landmarks instantly using keyboard shortcuts, rather than having to listen to every element on the page in order. A page built with `<div>` tags provides no landmarks and is effectively inaccessible. From a developer experience perspective, semantic HTML is self-documenting. A developer reading the source code of a MicroVest page immediately understands the page structure from the element names alone, without needing code comments. MicroVest uses the full suite of HTML5 semantic elements on every page: `<header>` wraps the sticky navigation, `<main>` wraps the primary page content, `<section>` groups thematically related blocks within main, `<article>` wraps self-contained portfolio stat cards, and `<footer>` holds the copyright notice and disclaimer.

4.3 Document Structure Tags

The `<html lang='en' data-theme='dark'>` root element performs two functions beyond marking the document root. The `lang='en'` attribute tells screen readers and browser translation tools the primary language of the document, enabling correct pronunciation rules in screen reader speech synthesis. The `data-theme='dark'` attribute is a custom HTML5 data attribute that serves as the anchor for MicroVest's entire CSS theming system. When the user clicks the theme toggle button, JavaScript executes `document.documentElement.setAttribute('data-theme', 'light')`,

and the CSS rule `:root[data-theme='light']` immediately activates, overriding all dark theme custom property values with light theme equivalents — causing the entire page to re-render in a new colour scheme without any JavaScript directly touching styles.

The `<head>` element is the non-rendered metadata container of every MicroVest page. Its contents are processed by the browser but never directly displayed to the user. Inside `<head>`, the meta `charset='UTF-8'` declaration specifies the character encoding for the document. UTF-8 is the universal encoding standard that supports all Unicode characters, including the Indian Rupee symbol (Rs.), mathematical symbols, and all international alphabets. Without this declaration, some browsers might misinterpret the byte sequences and display garbled characters. The `meta name='viewport' content='width=device-width, initial-scale=1.0'` declaration is essential for responsive design. Without it, mobile browsers render the page at a simulated desktop width of around 980 pixels and then scale the entire page down to fit the physical screen — making all text and elements appear tiny. With this declaration, the browser uses the device's actual pixel width as the viewport width, ensuring MicroVest's responsive CSS media queries can function as intended.

The `<link rel='preconnect'>` tags that appear before the Google Fonts stylesheet links are performance optimisation declarations. When the browser parses a preconnect link, it immediately begins the TCP/IP connection handshake with the specified domain, establishing the connection in the background before the fonts are actually needed. By the time the subsequent `<link rel='stylesheet'>` requests the font CSS from `fonts.googleapis.com`, the connection is already open, eliminating the connection establishment time from the critical rendering path. The `<link rel='stylesheet'>` tags load CSS files in a deliberate order: `variables.css` is always listed first because all other CSS files use `var(--token-name)` to reference its custom properties. If any file that references CSS variables were to load before `variables.css`, all `var()` calls would resolve to their initial value (empty) and the styles would not apply. Finally, all JavaScript `<script>` tags in MicroVest are placed immediately before the closing `</body>` tag. This positioning ensures that the entire HTML document has been parsed and all DOM elements exist in memory before any JavaScript executes. Scripts placed in `<head>` without the `'defer'` attribute would run before DOM elements are created, causing `querySelector` and `getElementById` calls to return null.

4.4 Semantic Landmark Tags

The `<header>` element is used on all four MicroVest pages and serves as the banner landmark region. It contains the site logo (a stylised MV monogram and the 'MicroVest' text), the navigation link group, and the theme toggle button. Screen readers announce this element as the 'banner' landmark, allowing users to immediately identify and navigate to the site identity area. The header is styled with `position: sticky` and `top: 0`, meaning it remains visible at the top of the viewport as the

user scrolls through the page content below it. A backdrop-filter: blur(24px) is applied to the navbar, creating a frosted glass effect as page content scrolls behind it.

The `<nav>` element wraps the navigation link group inside the header. It contains anchor tags pointing to all four pages, plus the theme toggle button. The `<nav>` landmark is recognised by all screen readers and announced as the 'navigation' region. A user who navigates by landmarks can press a single keystroke to jump directly to the navigation and find the link they want, without tabbing through the logo, hero content, or anything else. The `<main>` element appears exactly once per page — HTML5 specifies that a document must not have more than one non-hidden `<main>`. On `stocks.html`, `<main>` contains the page hero, the filter bar section, and the stocks grid section. On `portfolio.html`, it contains the summary cards strip, the allocation section, and the holdings table section.

The `<section>` element groups thematically related content within `<main>`. MicroVest's usage of `<section>` is always accompanied by an `aria-label` attribute: for example, `<section class='filter-bar' aria-label='Stock filters'>`. Without the `aria-label`, the section would be an unnamed landmark — technically valid but unhelpful to screen readers. With the `aria-label`, the landmark is announced as 'Stock filters region' when a screen reader user navigates to it, providing immediate orientation. The `<article>` element is used specifically on `portfolio.html` for each of the four summary stat cards: Total Value, Amount Invested, Total Gain/Loss, and Gain/Loss Percentage. An `<article>` represents a discrete, self-contained unit of content that would be meaningful if extracted from its context. Each portfolio stat card satisfies this criterion independently. The `<footer>` element is present on all four pages and contains the copyright notice and educational disclaimer. It constitutes the 'contentinfo' landmark, allowing screen reader users to jump directly to footer information.

Heading elements `<h1>` through `<h3>` are used throughout MicroVest following a strict hierarchy. Each page contains exactly one `<h1>` element — the primary page title that defines the document's topic. On `stocks.html` the `h1` reads 'Explore the Market'. On `portfolio.html` it reads 'My Portfolio'. This uniqueness is important because screen readers present the `h1` as the document title, and search engines treat it as the primary keyword signal. `<h2>` elements head each major content section within a page. `<h3>` elements head subsections or individual items — such as the stock company name inside each stock card, or individual glossary terms on `education.html`. This hierarchy creates a logical document outline that can be navigated by screen reader users pressing the H key to jump between headings.

4.5 Form & Input Elements

The buy calculator panel on `stocks.html` is the most form-intensive component in MicroVest. It contains an `<input type='number'>` element for the investment amount, styled and configured to provide immediate client-side validation. The `type='number'` attribute activates the browser's native numeric input mode: on mobile devices, the

operating system keyboard switches to a numeric keypad automatically. The `min='10'` attribute prevents the user from entering a value below Rs. 10 (the minimum investment). The `max='1000000'` attribute caps the entry at Rs. 10 lakh. The `step='0.01'` attribute allows fractional amounts like Rs. 25.50. The `placeholder='Enter amount (min Rs.10)'` attribute displays hint text inside the field before the user begins typing. The browser enforces all these constraints natively — if the user types a number below 10 and submits, the browser flags the field as invalid without any JavaScript validation code needed.

The search input uses `type='search'` rather than the generic `type='text'`. This semantic distinction matters: many browsers add a clear button (an X icon) automatically to search inputs, allowing users to clear their query with one click. On macOS Safari, search inputs also receive rounded corners to match platform conventions for search fields. The `id='stock-search'` attribute links this input element to its associated `<label>` element via the label's `for='stock-search'` attribute. This linkage means clicking the label text focuses the input field — a critical accessibility feature for users who rely on larger click targets. The sort `<select>` element creates a native browser dropdown containing six `<option>` elements: Default Order, Price: Low to High, Price: High to Low, Top Gainers, Top Losers, and Name A-Z. Native `<select>` elements are fully keyboard-accessible by default and are announced correctly to screen readers, avoiding the accessibility pitfalls of custom JavaScript dropdown implementations. The `<button>` element is used for every interactive control in MicroVest — it is natively focusable via the Tab key, natively activatable via Enter or Space, and is announced to screen readers as an interactive button.

4.6 Table Elements

The holdings table on `portfolio.html` is the primary use of semantic table markup in MicroVest. For this data — stock name, number of shares, average buy price, current price, current value, and gain/loss per holding — a `<table>` is the semantically correct choice. Tabular data has inherent row-column relationships: each row represents one holding, and each column represents one attribute of that holding. Rendering this data as a series of styled divs would lose the semantic relationship between headers and data cells, breaking screen reader accessibility. The `<table>` wrapper is further divided into `<thead>` for the header row and `<tbody>` for the data rows. This distinction allows browsers to handle printing correctly: when a table spans multiple printed pages, the browser can repeat the `<thead>` content at the top of each page, ensuring the column headers remain visible throughout. JavaScript in `portfolio.js` targets `document.getElementById('holdings-tbody')` to dynamically inject and update rows as the portfolio changes. The `<th>` elements in the header row have implicit scope attributes that associate each header with its column, enabling screen readers to announce 'Shares: 0.2534' rather than just '0.2534' when navigating cells. Price cells use `font-family: var(--font-mono)` to render numbers in Space Mono, preventing layout shift as digits change during live price updates.

4.7 Inline SVG — Charts Without Libraries

SVG (Scalable Vector Graphics) is an XML-based vector image format that can be written directly inside HTML5 and becomes part of the document's DOM, accessible to CSS and JavaScript. MicroVest uses inline SVG for two chart types: sparkline charts on each stock card (tiny line graphs showing the last ten price movements), and a value trend chart on portfolio.html. The SVG root element uses `viewBox='0 0 100 30'` which defines an internal coordinate system 100 units wide by 30 units tall. The actual rendered size is controlled entirely by CSS (width: 100%, height: 40px on stock cards), and the browser automatically scales the `viewBox` to fit this container. This means the SVG is fully responsive — it fills exactly the available width on any screen size without any JavaScript resizing code.

The sparkline is drawn using a `<polyline points='... '>` element. A polyline connects a series of x,y coordinate pairs with straight line segments. JavaScript computes new coordinate pairs every two seconds: the x-coordinate represents the point's position along the time axis (mapped from 0 to 100 across the chart width), and the y-coordinate represents the price (normalised to the 30-unit height of the `viewBox`). The polyline's `points` attribute is updated via `svg.querySelector('polyline').setAttribute('points', newPoints)` — a direct DOM attribute manipulation that the browser handles efficiently without requiring a full re-render. The portfolio trend chart adds a filled area below the line using a `<path d='... '>` element, whose `d` attribute contains SVG path commands: `M` (moveto) positions the starting point, `L` (lineto) draws line segments through each data point, and `Z` (closepath) closes the shape back to the baseline. A `<defs>` block containing a `<linearGradient>` definition creates a gradient fill — cyan (`#00D4FF`) at full opacity at the top, fading to fully transparent at the bottom — which the path references via `fill='url(#trendGradient)'`. The entire charting system requires no external libraries: it is implemented in approximately 30 lines of JavaScript and native SVG markup.

4.8 HTML Attributes — Standard & Custom data-*

HTML attributes provide additional information about elements. Standard attributes used throughout MicroVest include `id` (unique element identifier for JavaScript targeting), `class` (CSS styling hook), `href` (link destination), `src` (resource source), `type` (input or button behaviour), `placeholder` (hint text), `min/max/step` (numeric input constraints), `hidden` (boolean visibility control), and the ARIA attributes described in the next section. HTML5 introduced a formal system for custom attributes: any attribute whose name begins with `'data-'` is a valid custom data attribute, and its value is preserved exactly as written. These attributes do not affect rendering or the element's default behaviour — they exist solely to embed application-specific metadata in HTML elements.

MicroVest uses `data-*` attributes extensively as JavaScript hooks, deliberately separating them from CSS class names. Each stock card is rendered with `data-ticker='RELIANCE'` (or the appropriate ticker symbol). When the PriceSimulator

fires an update for RELIANCE, `stocks.js` executes `document.querySelector('[data-ticker="RELIANCE"]')` to locate that specific card in $O(1)$ time without iterating through an array or relying on a fragile DOM index. Each risk filter button carries `data-risk='low'`, `data-risk='medium'`, or `data-risk='high'`. When clicked, the event handler reads `event.target.dataset.risk` (the dataset API automatically converts `data-risk` to `dataset.risk`) to determine which filter to apply. Each calculator panel carries `data-calc='RELIANCE'`, allowing `openCalculator()` to locate the correct panel using `document.querySelector('[data-calc="'+ticker+""]')`. The landing page ticker items carry `data-preview='RELIANCE'` for the same reason. The advantage of using `data-*` attributes over CSS classes for this purpose is conceptual clarity: CSS classes communicate styling intent, while `data-*` attributes communicate data relationships. Mixing the two risks a CSS class rename silently breaking JavaScript logic.

4.9 ARIA Accessibility Attributes

Accessible Rich Internet Applications (ARIA) is a W3C specification that extends HTML with attributes designed to make dynamic web content and custom UI components accessible to users with disabilities. ARIA attributes do not change an element's visual appearance or default behaviour — they communicate semantic information directly to assistive technology such as screen readers. The most important rule of ARIA use is 'No ARIA is better than bad ARIA': incorrectly applied ARIA can make a page less accessible than using no ARIA at all. MicroVest applies ARIA precisely and only where native HTML semantics are insufficient.

The `aria-label` attribute provides a spoken name for elements that have no visible text label. The theme toggle button in MicroVest displays only a moon or sun emoji icon — an icon that is visually intuitive but meaningless to a screen reader. Without `aria-label='Toggle theme'`, the screen reader would either announce 'button' (no label at all) or attempt to describe the emoji character. With the `aria-label`, it announces 'Toggle theme, button', which is perfectly clear. Similarly, the filter button group container carries `aria-label='Filter stocks by risk level'` on its `role='group'` wrapper. The `aria-expanded='true/false'` attribute is added to each accordion trigger button on `education.html`'s glossary section. When a user presses an accordion button to expand a term's definition, JavaScript sets `btn.setAttribute('aria-expanded', 'true')`. The screen reader then announces 'expanded' alongside the button label, informing the user that content has been revealed below. When the accordion closes, `aria-expanded` is set to 'false' and the screen reader announces 'collapsed'. The `aria-hidden='true'` attribute is applied to the aurora background `<div>` on every page. This element is purely decorative and contains no meaningful content. Without `aria-hidden`, a screen reader would enter this element and either announce it as an empty region or attempt to read its CSS content, which is confusing and disruptive. With `aria-hidden='true'`, the element is completely invisible to the accessibility tree.

4.10 Complete HTML Tag Reference Table

Tag	Category	Used In	Semantic Meaning & Purpose in MicroVest
<!DOCTYPE html>	Declaration	All pages	Activates HTML5 standards mode in all browsers
<html lang='en' data-theme='dark'>	Root	All pages	Root element; lang enables correct screen reader pronunciation; data-theme drives the entire CSS theming system
<head>	Metadata container	All pages	Non-rendered section for all metadata, stylesheets, and font imports
<meta charset='UTF-8'>	Metadata	All pages	UTF-8 encoding supports the Rs (rupee) symbol and all international characters
<meta name='viewport'>	Metadata	All pages	Enables responsive layout; tells mobile browsers to use device width, not a simulated desktop
<meta name='description'>	Metadata	All pages	SEO description shown in Google search results
<link rel='preconnect'>	Resource hint	All pages	Pre-establishes TCP connection to Google Fonts CDN before stylesheet is requested — reduces font load latency
<link rel='stylesheet'>	Resource	All pages	Loads CSS in cascade order: variables -> base -> components -> page CSS
<title>	Metadata	All pages	Browser tab title and bookmark label
<body class='page-*'>	Root content	All pages	class='page-stocks' etc. enables page-specific CSS overrides without !important
<div class='aurora-bg' aria-hidden='true'>	Container	All pages	Decorative animated gradient; aria-hidden excludes it from screen readers
<header>	Landmark	All pages	Banner landmark; contains the navigation and site identity
<nav>	Landmark	All pages	Navigation landmark; contains Stocks / Portfolio / Learn links + theme toggle

<a href>	Inline	All pages	Navigation links between pages; relative paths keep site portable
<main>	Landmark	All pages	Primary content landmark; one per page; screen readers jump here via main content shortcut
<section aria-label>	Sectioning	All pages	Thematic content grouping; aria-label names the landmark region
<article>	Sectioning	portfolio.html	Self-contained stat card (Total Value, Invested, Gain/Loss, Gain%)
<h1>	Heading	All pages	One per page — primary page topic for SEO and screen reader outline
<h2>	Heading	All pages	Major section headings (e.g. Explore Stocks, My Portfolio)
<h3>	Heading	All pages	Subsection headings (stock names in cards, glossary terms)
<p>	Text	All pages	Standard paragraphs for body text, descriptions, subtitles
	Inline	All pages	Inline wrapper for specific styling (price values, change%, ticker symbol, badge text)
<button>	Interactive	All pages	Keyboard-accessible interactive control; used for Invest Now, Confirm Purchase, filter buttons, theme toggle
<input type='number'>	Form	stocks.html	Investment amount entry; min='10' max='1000000' step='0.01' provide built-in validation
<input type='search'>	Form	stocks.html	Stock search field; semantic type enables browser clear button
<select>	Form	stocks.html	Sort dropdown with 6 options (Default, Price High/Low, Top Gainers/Losers, Name A-Z)
<option>	Form	stocks.html	Individual sort choices inside the <select>
<table>	Table	portfolio.html	Semantic tabular data for the holdings list

<thead>	Table	portfolio.html	Groups header row; browser repeats on print across multiple pages
<tbody id='holdings-tbody'>	Table	portfolio.html	Target for JavaScript dynamic row injection
<tr>	Table	portfolio.html	One row per holding (or one header row)
<th>	Table	portfolio.html	Column header with implicit scope attribute
<td>	Table	portfolio.html	Data cells for shares, prices, values, gain/loss
<svg viewBox>	Graphics	stocks.html, portfolio.html	Inline SVG container for sparkline and trend charts; no external library needed
<polyline points>	SVG	stocks.html	Multi-point price line; points attribute updated by JavaScript every 2 seconds
<path d>	SVG	portfolio.html	Area fill path below trend line; d attribute contains M/L/Z path commands
<defs> / <linearGradient>	SVG	portfolio.html	Gradient fill definition for trend area (cyan to transparent)
<footer>	Landmark	All pages	Footer landmark; copyright and disclaimer
<script src>	Resource	All pages	Loads JS files; placed before </body> so DOM exists when scripts run
data-ticker (attribute)	Custom data	stocks.html	Embeds stock identity in DOM; JS queries [data-ticker='RELIANCE'] without fragile index
data-risk (attribute)	Custom data	stocks.html	Risk value on filter buttons; JS reads btn.dataset.risk to apply filter
data-calc (attribute)	Custom data	stocks.html	Calculator panel identifier; openCalculator() queries [data-calc='\${ticker}']
data-preview (attribute)	Custom data	index.html	Ticker identity on landing page preview items
aria-label	ARIA	All pages	Provides spoken label for elements without visible text (theme toggle, filter group)
aria-expanded	ARIA	education.html	Communicates accordion open/closed state to screen readers
aria-hidden	ARIA	All pages	Hides decorative elements (aurora

			background) from screen reader output
role='group'	ARIA	stocks.html	Groups the risk filter buttons as a semantic unit for screen readers
hidden (attribute)	Boolean attr	portfolio.html	Hides empty-state message when portfolio has holdings; shown via JS when portfolio is empty

5. CSS3 — Concepts, Properties & Usage in MicroVest

5.1 What is CSS3?

Cascading Style Sheets (CSS) is the language that controls every visual aspect of a web page — colours, typography, spacing, layout, visual effects, animations, and responsive behaviour. CSS describes how HTML elements should be rendered on screen, in print, or in other media. CSS is called a 'styling language' rather than a programming language because it consists of declarative rules — property-value pairs that instruct the browser how to render a matched element — rather than executable logic. CSS3 is the current evolution of CSS, developed as a set of modular specifications that extend CSS2 with powerful new capabilities. Key CSS3 modules include CSS Custom Properties (variables), Flexible Box Layout (Flexbox), Grid Layout, CSS Transforms, CSS Transitions, CSS Animations and @keyframes, Media Queries for responsive design, Calc() for computed values, Clamp() for fluid sizing, and advanced filter effects including backdrop-filter.

The word 'Cascading' is the most fundamental concept in CSS. When multiple CSS rules target the same element with the same property, the browser must resolve the conflict using a formal algorithm called the cascade. The cascade evaluates three factors in order. First, Origin: styles written by the page developer (called author styles) override styles built into the browser (called user-agent styles). Second, Specificity: more specific selectors override less specific ones regardless of order — an ID selector (such as #holdings-tbody) is more specific than a class selector (such as .glass-card), which is more specific than an element selector (such as div). Third, Source Order: when two rules have equal specificity, the rule that appears later in the CSS source overwrites the earlier one. This is why MicroVest loads CSS files in a precise order: variables.css first (lowest specificity, foundational tokens), then base.css (element selectors), then components.css (class selectors), then each page's own CSS file (more specific class selectors). Each layer builds on the layer below it without needing to use !important hacks to override earlier rules.

A foundational architectural decision in MicroVest is that CSS is completely separated from HTML. There are no inline style='...' attributes on any HTML element, and no <style> tags embedded in any HTML file. Every styling rule lives in one of the seven CSS files. This practice, known as Separation of Concerns, ensures that design changes can be made by editing CSS files without touching HTML, and that HTML files remain clean, readable, and free of presentation logic. It also means that a single CSS change — such as updating the primary accent colour in variables.css — propagates consistently to every page that uses that variable, rather than requiring a find-and-replace operation across multiple HTML files.

5.2 CSS Custom Properties (Variables)

CSS Custom Properties — commonly called CSS Variables — are one of the most powerful features introduced in CSS3. A custom property is declared using a double-dash prefix followed by any name: `--property-name: value`. It is referenced using the `var()` function: `color: var(--accent-cyan)`. Unlike preprocessor variables found in SASS or LESS (which are compiled away before the browser ever sees the CSS), CSS custom properties exist at runtime in the browser. This has profound implications: they can be read and written by JavaScript, they participate in the cascade and are inherited by child elements, and they can be overridden at any level of the DOM hierarchy. A custom property set on `:root` is inherited by every element in the document, but the same property can be overridden for a specific element or class — and only that element's subtree will see the new value.

In MicroVest, all custom properties are declared on the `:root` pseudo-element in `variables.css`. `:root` is a CSS pseudo-class that matches the highest-level element in the document — the `<html>` element. Because it is the root of the document tree, all custom properties declared here cascade to every other element automatically. MicroVest's `variables.css` defines over 50 custom properties covering the complete design system: colour tokens (`--accent-cyan: #00D4FF`, `--up-strong: #00E676`, `--down-strong: #FF3D57`), typography tokens (`--font-display: 'Syne' sans-serif`, `--font-body: 'DM Sans' sans-serif`, `--font-mono: 'Space Mono' monospace`), spacing tokens (`--space-2: 0.5rem` through `--space-12: 3rem`), border radius tokens (`--radius-sm: 8px`, `--radius-xl: 16px`), glass effect tokens (`--glass-bg: rgba(15,22,33,0.75)`), and animation timing tokens (`--transition-base: 0.25s ease`).

CSS — *variables.css*

```
/* variables.css - Design token declarations */

:root {

  --accent-cyan:      #00D4FF;

  --accent-violet:    #7B61FF;

  --font-display:     'Syne', sans-serif;

  --space-4:          1rem;

  --radius-xl:        16px;

  --transition-base:  0.25s ease;

  --glass-bg:         rgba(15, 22, 33, 0.75);

}
```

The most powerful demonstration of CSS custom properties in MicroVest is the dark/light theming system. A second block in `variables.css` uses an attribute selector

to override a subset of the colour and glass variables when the `data-theme='light'` attribute is present on the `<html>` element. When JavaScript executes `document.documentElement.setAttribute('data-theme', 'light')`, every element in the page that references any of the overridden variables instantly re-renders with the new values. The background changes from near-black to pale blue, text changes from off-white to near-black, and glass cards change from dark-tinted to light-frosted — all with a CSS transition-based smooth fade. This entire theme switch is accomplished with one line of JavaScript because the CSS foundation was designed for it from the start.

CSS — *variables.css* (light theme override)

```
:root[data-theme="light"] {  
  
  --bg-base:      #F0F4FF;  
  
  --bg-surface:   #FFFFFF;  
  
  --text-primary: #0D1117;  
  
  --glass-bg:     rgba(255, 255, 255, 0.70);  
  
}
```

5.3 The Cascade and Specificity

Specificity is a numerical score that the browser assigns to each CSS selector to determine which rule wins when multiple rules target the same element and property. The scoring system uses three categories. The first category covers ID selectors (`#id-name`), each worth 100 specificity points. The second covers class selectors (`.class-name`), attribute selectors (`[data-risk]`), and pseudo-classes (`:hover`, `:root`), each worth 10 points. The third covers element type selectors (`div`, `p`, `header`) and pseudo-elements (`::before`, `::after`), each worth 1 point. The universal selector (`*`) contributes 0 points. These scores are applied independently and do not carry over: no number of class selectors can outweigh even one ID selector.

In MicroVest's CSS architecture, the load order is designed to respect this specificity hierarchy. `variables.css` declares custom properties on `:root` — specificity score of 10 (pseudo-class). `base.css` uses element selectors (`*`, `body`, `header`, `nav`) with specificity scores of 0-1. `components.css` uses class selectors (`.glass-card`, `.btn-primary`) with specificity scores of 10-20. `stocks.css` uses compound selectors like `.stock-card:hover` (specificity 20) and `.stock-card.price-up` (specificity 20) — paired classes that are more specific than single-class rules in `components.css`, allowing page-specific hover states and animation states to override the base component styles without conflicts. The rule `#holdings-tbody` (specificity 100) in `portfolio.css` ensures that the dynamically injected table body is targeted uniquely and cannot be accidentally overridden by any class-level rule.

5.4 Box Model

Every HTML element in a web page is rendered as a rectangular box. The CSS Box Model defines how the total size and spacing of this box is calculated. The box consists of four concentric layers: the content area (where text and child elements are rendered, sized by width and height properties), the padding (transparent space between the content and the border, controlled by padding-top/right/bottom/left), the border (a visible or invisible line around the padding, controlled by border-width, border-style, border-color), and the margin (transparent space outside the border that separates this element from adjacent elements). By default, CSS calculates width and height to mean only the content area — padding and border are added on top, making the actual space the element occupies larger than specified. This counter-intuitive behaviour causes layout maths errors. MicroVest applies box-sizing: border-box globally via `*, *::before, *::after { box-sizing: border-box; }` in `base.css`. With border-box, the width property defines the total width including padding and border — a `.glass-card` with `width: 300px` is exactly 300px wide on screen, regardless of its padding or border thickness.

5.5 Flexbox

CSS Flexible Box Layout (Flexbox) is a one-dimensional layout system that arranges elements along either a horizontal row or a vertical column. A parent element becomes a flex container by declaring `display: flex`. Its direct children become flex items that can grow, shrink, and align in relation to each other and to the container. Flexbox is ideal for UI components that need to distribute space among a set of items along a single axis — headers, navigation bars, card footers, and button groups. The MicroVest navbar is the most visible Flexbox usage: the `nav-container` uses `display: flex` with `justify-content: space-between` to push the logo to the left end and the navigation controls to the right end, and `align-items: center` to vertically centre all items at the midpoint of the navbar height. This three-property combination — `display flex`, `space-between`, `center` — is the fundamental Flexbox navbar pattern used in virtually all modern web applications.

Additional Flexbox properties used throughout MicroVest: `gap` creates consistent spacing between flex items without needing margin hacks. The navigation link group uses `gap: 2rem` between links. `flex-wrap: wrap` allows flex items to overflow to a new row when the container is too narrow. The hero CTA button group and the stats bar on the landing page use `flex-wrap: wrap` so they collapse to a column layout on narrow mobile screens without any media query needed. `flex: 1` on a flex item makes it grow to fill all remaining space in the container — used on the search input so it expands to fill the available width in the filter bar. `flex-shrink: 0` prevents an item from shrinking below its natural size even when the container is too small for all items — used on the theme toggle button so it never becomes too small to click. `flex-direction: column` changes the main axis to vertical — used in media queries to stack hero elements vertically on mobile.

5.6 CSS Grid

CSS Grid Layout is a two-dimensional layout system that arranges elements into rows and columns simultaneously. A parent element becomes a grid container by declaring `display: grid`. Its children become grid items that are placed into the grid cells automatically or explicitly. CSS Grid is fundamentally different from Flexbox: Flexbox works in one dimension at a time (either a row or a column), while Grid works in both dimensions simultaneously, making it ideal for page-level layouts and card grids where both row alignment and column alignment matter.

The stocks grid on `stocks.html` uses `display: grid` combined with `grid-template-columns: repeat(3, 1fr)`. The `repeat(3, 1fr)` notation instructs the browser to create three columns, each taking one fraction of the available container width. The 'fr' unit (fraction) is unique to CSS Grid — it represents a share of the remaining space after fixed-width and auto-width columns have been allocated. With three 1fr columns and a container width of 900px, each column is 300px wide (minus the gap). Row gap and column gap between grid items are controlled by the `gap: 1.5rem` shorthand on the grid container. This spacing is applied uniformly in both dimensions with a single property. The featured stocks section on the landing page uses `grid-template-columns: repeat(4, 1fr)` for a four-column arrangement. As viewport width decreases, media queries change these column definitions: at `max-width 1024px` the stocks grid switches to `repeat(2, 1fr)` (two columns), and at `max-width 768px` it switches to a single column (`grid-template-columns: 1fr`). Grid was chosen over Flexbox for these card layouts because the grid enforces two-dimensional alignment — every card in a row is the same height, and every card in a column lines up in the same column position — which is not achievable with Flexbox alone.

5.7 Positioning

CSS provides five position values that control how an element is placed in the document flow. `'position: static'` is the default — elements follow normal document flow. `'position: relative'` moves an element from its normal position using `top`, `right`, `bottom`, or `left` offsets, but the element still occupies its original space in the flow. Importantly, `position: relative` also creates a positioning context for absolutely positioned children. MicroVest uses `position: relative` on stock cards so that the calculator panel — which is `position: absolute` — is positioned relative to the card rather than the viewport. `'position: absolute'` removes an element from normal flow and positions it relative to its nearest positioned ancestor. The step number badge in the How It Works section uses `position: absolute` with `top: -12px` and `right: -12px` to position it in the top-right corner of its card. `'position: sticky'` keeps an element in normal flow until the user scrolls to a threshold, at which point it 'sticks' to the viewport. The MicroVest header uses `position: sticky` with `top: 0`, making it remain anchored to the top as the user scrolls. `'position: fixed'` removes an element from flow and positions it relative to the viewport — it stays at the same position even when the page scrolls. The aurora background overlay uses `position: fixed` with

`inset: 0` (shorthand for `top: 0; right: 0; bottom: 0; left: 0`) to cover the entire viewport at all times, and `z-index: -1` to ensure it sits behind all page content.

5.8 CSS Animations & Transitions

CSS transitions provide smooth interpolation between two states of a CSS property. When an element's property changes — due to `:hover`, `:focus`, or a JavaScript class toggle — instead of snapping instantly to the new value, the browser animates the change over a specified duration using a specified easing function. The stock cards in MicroVest use a multi-property transition that activates on hover: `transform` transitions over 0.28 seconds, `box-shadow` over 0.28 seconds, and `border-color` over 0.28 seconds. The easing function `cubic-bezier(0.34, 1.56, 0.64, 1)` is a spring curve: the second control point value of 1.56 (greater than 1.0) creates an overshoot — the card briefly lifts more than its final `-7px translateY` target before settling, creating a natural, bouncy feel. The hover shadow changes from a neutral ambient shadow to a directional composition: a faint cyan glow above and to the sides, and a deep dark shadow below, creating the visual illusion that the card is floating in space.

CSS — *stocks.css* (hover animation)

```
.stock-card {  
  transition: transform 0.28s cubic-bezier(0.34, 1.56, 0.64, 1),  
             box-shadow 0.28s ease,  
             border-color 0.28s ease;  
}  
  
.stock-card:hover {  
  transform: translateY(-7px);  
}
```

CSS `@keyframes` animations define multi-step animated sequences that the browser executes automatically once triggered. MicroVest defines three key animations. The 'aurora-shift' animation applies to the aurora background overlay and animates over 12 seconds, infinitely, alternating direction each cycle. It shifts `background-position` and applies a subtle transform: `scale()` to create the visual effect of the gradient nebula slowly drifting across the page. The 'price-flash-up' and 'price-flash-down' animations are triggered by JavaScript adding the `.price-up` or `.price-down` CSS class to a stock card whenever the price changes. The card border colour transitions from its resting cyan (`#00D4FF` at 18% opacity) to a bright green (`#00E676` at 80% opacity) or red (`#FF3D57` at 80% opacity) at the 25% keyframe mark, then fades back to the resting state by 100%. Simultaneously, a coloured box-shadow creates a radiant glow effect. The entire animation lasts 0.6 seconds. A critical implementation detail: when JavaScript removes the `.price-up` class and

immediately re-adds it (because the price changed again before the animation finished), the browser may batch these changes and not re-trigger the animation since no actual state change occurred from its perspective. The solution is 'void card.offsetWidth' — accessing the `offsetWidth` property forces the browser to perform a synchronous layout recalculation (a 'reflow'), flushing the pending style changes and guaranteeing the animation restarts. The 'card-enter' animation creates a staggered entrance effect: each card is assigned a CSS custom property `--card-index` with its position index (0 through 9). The animation-delay is calculated as `'calc(var(--card-index) * 0.05s)'`, so card 0 enters at 0ms, card 1 at 50ms, card 2 at 100ms, and so on, creating a cascading wave of cards appearing.

CSS — *stocks.css* (price flash animation)

```
@keyframes price-flash-up {  
  
  0%    { border-color: rgba(0, 212, 255, 0.18); }  
  
  25%   { border-color: rgba(0,230,118,0.80);  
          box-shadow: 0 0 25px rgba(0,230,118,0.40); }  
  
  100%  { border-color: rgba(0, 212, 255, 0.18); }  
}
```

5.9 Glassmorphism & Visual Effects

Glassmorphism is a visual design trend that simulates the appearance of frosted or smoked glass: a semi-transparent surface that reveals a blurred version of whatever is behind it, combined with a subtle border and shadow to define the glass edge. The effect creates a sense of depth and material quality that makes UI components feel premium and dimensional against a rich background. In MicroVest, the aurora background exists precisely to be seen through the glass cards — the cyan and violet gradients drift behind the cards, creating a living, colourful backdrop that the glassmorphism effect renders into a blurred, saturated atmospheric glow.

CSS — *components.css* (glassmorphism)

```
.glass-card {  
  
  background: rgba(15, 22, 33, 0.75);  
  
  backdrop-filter: blur(20px) saturate(150%);  
  -webkit-backdrop-filter: blur(20px) saturate(150%);  
  
  border: 1px solid rgba(0, 212, 255, 0.18);  
  
  border-radius: 16px;
```



```
box-shadow: 0 8px 32px rgba(0,0,0,0.45),  
           inset 0 1px 0 rgba(255,255,255,0.06);  
}
```

Each property in the glass card rule contributes to the effect. The background `rgba(15,22,33,0.75)` is a deep navy at 75% opacity — the 25% transparency allows the aurora behind to be seen while keeping the card readable. `backdrop-filter: blur(20px) saturate(150%)` applies a pixel-level blur filter to whatever is rendered behind the card in the browser's compositing stack — this is the defining property of glassmorphism. The `blur()` function smooths the gradient underneath, and `saturate(150%)` intensifies those colours by 50%, making the cyan and violet of the aurora pop vividly through the frosted surface. The `-webkit-backdrop-filter` duplicate is a vendor prefix required for Safari and older iOS browsers that have not yet adopted the unprefixed standard. The border `rgba(0,212,255,0.18)` is a faint cyan outline that defines the card edge against the background — without it, the semi-transparent card would blend indistinguishably into the background. The inner box-shadow `'inset 0 1px 0 rgba(255,255,255,0.06)'` places a 1-pixel white line at the very top inside edge of the card, simulating light reflecting off the upper surface of a physical glass panel.

5.10 Responsive Design & Media Queries

Responsive web design is the practice of building layouts that adapt gracefully to any screen size — from a 375px wide smartphone to a 1440px wide desktop monitor — using a single codebase. The foundational tool for responsive design in CSS3 is the media query: a conditional rule block that applies its CSS declarations only when specified conditions about the viewport are met. MicroVest is built desktop-first: the primary CSS in each file describes the full desktop layout, and media query blocks using max-width conditions progressively simplify the layout for smaller screens. MicroVest is tested and optimised for five primary breakpoints: 1280px and above (full desktop with three-column grid), 1024px (small desktop or landscape tablet, reducing to two-column grid), 768px (portrait tablet, single column grid, stacked hero), 640px (large mobile), and 375px (compact mobile, adjusting font sizes and padding).

CSS — *stocks.css* (media queries)

```
/* stocks.css - Responsive grid adjustments */  
  
@media (max-width: 1024px) {  
  .stocks-grid {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```



```

    }
}
@media (max-width: 768px) {
    .stocks-grid {
        grid-template-columns: 1fr;
    }
    .hero-section {
        flex-direction: column;
    }
}
}

```

Beyond breakpoint-based media queries, MicroVest uses CSS's `clamp()` function for fluid typography that scales continuously between breakpoints without any jarring jumps. The landing page hero heading uses `font-size: clamp(2.5rem, 5vw, 4rem)`. `Clamp` takes three arguments: a minimum value (2.5rem — the font will never be smaller than this), a preferred value (5vw — 5 percent of the viewport width, making the font size proportional to the screen), and a maximum value (4rem — the font will never be larger than this). On a 375px phone, 5vw = 18.75px = 1.17rem, which is below the minimum, so the font is clamped to 2.5rem. On a 1440px desktop, 5vw = 72px = 4.5rem, which is above the maximum, so the font is clamped to 4rem. Between those extremes, the font scales fluidly proportional to the screen width, eliminating the visual jump that occurs with step-based media query font changes.

5.11 CSS Theming System (Dark/Light Mode)

MicroVest's dark/light theme toggle is powered entirely by CSS custom properties and a single HTML attribute. The theme state is stored in two places: in browser `localStorage` (the key 'mv-theme' with value 'dark' or 'light') for persistence across sessions, and in the `data-theme` attribute on the `<html>` element for the CSS selector to read. When any MicroVest page loads, its JavaScript immediately reads `localStorage.getItem('mv-theme')` and applies it with `document.documentElement.setAttribute('data-theme', saved)` before the browser has rendered any content. This immediate application is critical: if the theme were applied half a second after the page loaded, the user would see a brief flash of the default dark theme before the saved light theme applied — an experience known as FOUCC (Flash of Unstyled Content). By setting the attribute synchronously in script code that runs before the first paint, the correct theme is applied from the very first rendered frame, and no flash occurs.

5.12 Complete CSS Properties Reference Table

Property / Rule	Value in MicroVest	File	What It Does & Why Used
--accent-cyan (CSS variable)	#00D4FF	variables.css	Single source of truth for primary accent colour — change once, updates everywhere
:root[data-theme='light']	Override block	variables.css	Overrides dark theme tokens when data-theme='light' is set on <html> by JavaScript
box-sizing: border-box	* selector	base.css	Makes width include padding and border — predictable layout calculation
position: fixed; inset: 0; z-index: -1; pointer-events: none	.aurora-bg	base.css	Aurora covers full viewport, sits behind all content, never intercepts mouse clicks
animation: aurora-shift 12s infinite alternate	.aurora-bg	base.css	Slowly drifts the gradient; alternate reverses direction each cycle; infinite never stops
position: sticky; top: 0; z-index: 1000	header	base.css	Navbar sticks to viewport top while scrolling; z-index 1000 ensures it appears above cards
backdrop-filter: blur(24px)	header, nav	base.css	Frosted glass navbar — blurs page content scrolling behind it
display: flex; justify-content: space-between; align-items: center	.nav-container	base.css	Logo left, links centre, actions right — classic navbar layout
background: rgba(15,22,33,0.75)	.glass-card	components.css	Semi-transparent — aurora gradient shows through, creating depth
backdrop-filter: blur(20px) saturate(150%)	.glass-card	components.css	Blurs and colour-saturates content behind cards
inset 0 1px 0 rgba(255,255,255,0.06)	.glass-card box-shadow	components.css	Inner top highlight — simulates light striking glass edge
max-height: 0; overflow: hidden; transition: max-height 0.35s ease	.accordion-content	education.css	Smooth accordion — JS sets maxHeight to scrollHeight to open, 0 to close
display: grid; grid-template-columns: repeat(3, 1fr)	.stocks-grid	stocks.css	3-column equal-width grid for stock cards
@media (max-width: 1024px)	repeat(2, 1fr)	stocks.css	Reduces to 2 columns on tablet

@media (max-width: 768px)	1fr	stocks.css	Single column on mobile
transform: translateY(-7px)	.stock-card:hover	stocks.css	Lifts card upward on hover
cubic-bezier(0.34, 1.56, 0.64, 1)	.stock-card transition	stocks.css	Spring easing — card briefly overshoots then settles; 1.56 creates the bounce
0 -4px 20px rgba(0,212,255,0.12), 0 20px 50px rgba(0,0,0,0.7)	.stock-card:hover box-shadow	stocks.css	Directional: cyan glow on top/sides, deep dark shadow below — floating card illusion
@keyframes price-flash-up	Border cyan to green to cyan	stocks.css	Card border briefly glows green when price increases, animation lasts 0.6s
@keyframes price-flash-down	Border cyan to red to cyan	stocks.css	Card border briefly glows red when price decreases
animation-delay: calc(var(--card-index) * 0.05s)	.stock-card	stocks.css	Staggered entrance — card 0 enters immediately, card 9 enters 450ms later
--bar-width (CSS var on element)	style='--bar-width: 66.9%'	portfolio.css	JS sets this variable per-element; CSS width: var(--bar-width) animates the bar
font-family: var(--font-mono)	Price/number elements	components.css	Space Mono is monospaced — digits are all equal width, preventing layout shift during live price updates
font-size: clamp(2.5rem, 5vw, 4rem)	.hero-title	landing.css	Fluid typography — scales smoothly between min and max with no breakpoint jump
-webkit-background-clip: text; -webkit-text-fill-color: transparent	.gradient-text	landing.css	CSS gradient text — renders the brand gradient as the actual text fill colour
border: 1px solid rgba(0,212,255,0.18)	.glass-card	components.css	Subtle cyan border makes card visible against dark background; brightens to 0.40 on hover

6. JavaScript — Concepts, Patterns & Usage in MicroVest

6.1 What is Vanilla JavaScript?

Vanilla JavaScript refers to plain, native JavaScript written without any external framework, library, or transpiler. The term 'Vanilla' is used colloquially to distinguish raw browser JavaScript from the abstracted versions commonly encountered in modern development — React, Vue.js, Angular, Svelte, jQuery, Lodash, Axios, and hundreds of others. These tools are valuable productivity accelerators in production development, but they abstract away the underlying browser platform. A developer building with React, for instance, writes JSX components and declarative state — React's runtime translates this to actual DOM operations. A developer writing Vanilla JavaScript is working directly with the browser's native APIs: the Document Object Model API, the Event API, the Web Storage API, the Web Animations API, and the ECMAScript language features themselves. MicroVest is written entirely in Vanilla JavaScript to demonstrate complete command of these native APIs and to ensure that every technique, pattern, and algorithm can be explained and understood at the fundamental level — undistorted by framework conventions.

6.2 Object-Oriented Programming — PriceSimulator Class

Object-Oriented Programming (OOP) is a programming paradigm that organises code around objects — self-contained units that bundle data (called properties or fields) and behaviour (called methods). A class is a blueprint that defines the structure and behaviour of a category of object. When a class is instantiated with the 'new' keyword, it creates a concrete object (called an instance) that has its own copy of the class's fields and inherits its methods. ES6 introduced the 'class' keyword to JavaScript as clean syntactic sugar over the language's prototype-based inheritance system. ES2022 then introduced truly private class fields — fields prefixed with '#' that are syntactically enforced to be inaccessible from outside the class. This is true language-level encapsulation, unlike the older convention of prefixing private fields with '_underscore' (which was merely a naming convention with no enforcement).

MicroVest's PriceSimulator class is the architectural centrepiece of the JavaScript layer. It encapsulates the entire price simulation engine behind a minimal, clean public interface. Four private fields hold all internal state: '#stocks' is an array of live stock objects (each containing all STOCKS data plus a current live price), '#history' is a Map from ticker symbol to an array of the last 10 prices (used to draw sparkline charts), '#intervalId' stores the return value of setInterval (needed to cancel the interval later), and '#callbacks' is an array of subscriber functions registered by page scripts. None of these fields can be accessed or modified from outside the class — any attempt to access sim.#stocks from page JS would raise a SyntaxError at parse time.

JavaScript — *simulator.js* (*PriceSimulator* class)

```
class PriceSimulator {
  #stocks; #history; #intervalId; #callbacks;

  constructor(stocksData) {
    this.#stocks = stocksData.map(s => ({ ...s, price: s.basePrice }));
    this.#history = new Map(stocksData.map(s => [s.ticker, [s.basePrice]]));
    this.#intervalId = null;
    this.#callbacks = [];
  }

  start() { this.#intervalId = setInterval(() => this.#tick(),
SIM_INTERVAL_MS); }

  stop() { clearInterval(this.#intervalId); }

  subscribe(fn) { this.#callbacks.push(fn); }

  getCurrentPrices() { return new Map(this.#stocks.map(s => [s.ticker,
s.price])); }

  getStock(ticker) {
    const s = this.#stocks.find(s => s.ticker === ticker);
    return s ? { ...s } : null; // shallow copy — external code cannot mutate
#stocks
  }

  #tick() { /* calls #calcNewPrice for each stock, updates #history,
notifies callbacks */ }

  #calcNewPrice(s) { /* GBM algorithm — see Section 6.10 */ }
}
```

The public interface is deliberately minimal: `start()`, `stop()`, `subscribe(fn)`, `getCurrentPrices()`, and `getStock(ticker)`. This minimal surface area enforces the Open/Closed Principle — the class is open for extension (new subscribers can be added) but closed for modification (page scripts cannot change simulation logic). The `getStock()` method returns a spread copy of the internal stock object (`{ ...s }`) rather

than a reference to the actual object. If a reference were returned, external code could write `'sim.getStock("TCS").price = 9999'`, directly mutating the private `'#stocks'` array through the object reference and corrupting the simulation. The spread copy prevents this by creating a new object with the same values — modifying the copy has no effect on `'#stocks'`.

6.3 The Observer Pattern — `subscribe()`

The Observer design pattern is one of the most widely used patterns in software engineering. It defines a one-to-many dependency relationship between objects: a single Subject maintains a list of Observers, and when the Subject's state changes, it automatically notifies all Observers by calling a method on each. The pattern is also known as the Publisher-Subscriber or Event Bus pattern. The key characteristic of the Observer pattern is decoupling: the Subject (publisher) knows nothing about the specific identity or implementation of its Observers. It only knows that each Observer has registered a callback function to be called when state changes.

In MicroVest, PriceSimulator is the Subject. `stocks.js` and `portfolio.js` are Observers. When `stocks.js` initialises on `stocks.html`, it calls `'sim.subscribe(onPriceUpdate)'` where `onPriceUpdate` is a function that receives `(ticker, price, changePct)` and updates the corresponding stock card in the DOM. When `portfolio.js` initialises on `portfolio.html`, it calls `'sim.subscribe(onPriceUpdate)'` where its `onPriceUpdate` function updates the holdings table and portfolio totals. Both files subscribe to the same PriceSimulator with different callback functions. The PriceSimulator does not need to know which page it is running on, or what the callbacks do — it simply calls `'this.#callbacks.forEach(fn => fn(ticker, price, pct))'` every 2 seconds. This decoupling means that adding a new page (say, a market-movers dashboard) would require only creating a new JS file that calls `sim.subscribe()` — no changes to `simulator.js` are needed. The Observer pattern in MicroVest directly mirrors the web browser's own `EventTarget/addEventListener` architecture, where the DOM element is the Subject and event listener functions are the Observers.

6.4 DOM Selection & Manipulation

The Document Object Model (DOM) is the browser's internal representation of an HTML document as a tree of node objects. Every HTML element becomes a node in this tree, with parent-child relationships matching the HTML nesting structure. JavaScript interacts with the page entirely through the DOM API — reading and writing properties of these node objects, which the browser then reflects in the rendered page. DOM manipulation is how JavaScript bridges the gap between data and visual output: when the PriceSimulator emits a new price for RELIANCE, `stock.js`'s callback must somehow get that number into the correct price element on screen, and it does so through DOM manipulation.

MicroVest uses four DOM selection methods. `document.getElementById('stock-search')` is the fastest selection method — since IDs are unique within a document, the browser maintains an internal ID index for

O(1) look-up. `document.querySelector('[data-ticker="RELIANCE"]')` uses CSS selector syntax to find the first DOM element matching the selector. This is used to look up a specific stock card by its data-ticker attribute when a price update arrives for that ticker. `document.querySelectorAll('[data-risk]')` returns a `NodeList` of all elements matching the selector — used to attach click listeners to all filter buttons. In terms of DOM mutation, `element.innerHTML = template` replaces an element's entire inner HTML with a new string — used by `renderStocks()` and `renderTable()` to rebuild grid sections or table rows from fresh data. `element.textContent = value` replaces text content only, without parsing HTML — safer than `innerHTML` for plain text updates as it provides natural protection against XSS injection. `element.classList.add('price-up')` adds a CSS class to trigger the flash animation; `classList.remove('price-up')` removes it. `element.style.setProperty('--bar-width', pct + '%')` sets a CSS custom property directly on a specific element's inline style, overriding the `:root` declaration only for that element — used to set individual allocation bar widths. `element.setAttribute('points', svgCoords)` updates the SVG polyline's coordinate string for sparkline charts.

6.5 Events & Event Delegation

Browser events are signals emitted by the browser or by user interactions that JavaScript can listen for using `element.addEventListener(eventType, handler)`. User interaction events include 'click', 'input', 'change', 'keydown', and 'mouseover'. Browser lifecycle events include 'DOMContentLoaded', 'load', 'scroll', and 'resize'. An event listener is a function that is called automatically by the browser whenever the specified event fires on the specified element.

MicroVest wraps all initialisation code in a 'DOMContentLoaded' event listener: `'document.addEventListener("DOMContentLoaded", () => { ... })'`. This event fires when the browser has finished parsing all HTML and constructing the full DOM tree, but before stylesheets, images, and other resources have finished loading. This is the correct moment to begin querying and manipulating DOM elements — all elements exist in the tree and can be found by `getElementById` and `querySelector`. Code that runs before `DOMContentLoaded` would find that these methods return null, because the elements have not yet been created. The filter buttons use `addEventListener('click')` attached directly to each button via `querySelectorAll` iteration. Each button's handler reads `'btn.dataset.risk'` (the dataset API automatically strips the 'data-' prefix and camelCases the result), updates the `currentFilter` variable, and calls `applyFiltersAndRender()`. The search input uses `addEventListener('input')` which fires on every keystroke — unlike 'change' which fires only when the field loses focus. Each input event updates `'searchQuery = e.target.value.toLowerCase()'` and re-renders. `window.addEventListener('scroll')` monitors the page scroll position and adds the class `'.scrolled'` to the header element when the user has scrolled more than 20px, increasing the navbar background opacity for better contrast over page content.

JavaScript — stocks.js (event setup)

```

document.addEventListener('DOMContentLoaded', () => {

  const searchInput = document.getElementById('stock-search');

  searchInput.addEventListener('input', (e) => {

    searchQuery = e.target.value.toLowerCase();

    applyFiltersAndRender();

  });

  document.querySelectorAll('[data-risk]').forEach(btn => {

    btn.addEventListener('click', () => {

      currentFilter = btn.dataset.risk;

      applyFiltersAndRender();

    });

  });

});

```

6.6 setInterval & the Price Simulation Loop

`setInterval(fn, ms)` is a browser-native timer function that repeatedly calls `fn` after every `ms` milliseconds until explicitly cancelled with `clearInterval(id)`. It returns a numeric identifier that refers to the scheduled interval. In `PriceSimulator`, `'this.#intervalId = setInterval(() => this.#tick(), SIM_INTERVAL_MS)'` is called in `start()`. The arrow function `() => this.#tick()` is critical: it preserves the lexical `'this'` binding from the surrounding class context. If `'this.#tick'` were passed directly (without wrapping in an arrow function), `'this'` inside `#tick` would be undefined in strict mode because `setInterval` calls functions with a different `'this'` context. The arrow function creates a closure that captures the surrounding `'this'` (the `PriceSimulator` instance), ensuring `#tick` can access all private fields correctly. `clearInterval(this.#intervalId)` in `stop()` cancels the interval. This is essential for memory management: without it, the interval would continue firing indefinitely even after the user navigates away, consuming CPU cycles and potentially causing errors as the callback tries to update DOM elements on a page that no longer exists.

6.7 ES6+ Features Used

Arrow functions (`=>`) provide a concise syntax for function expressions and, crucially, lexically bind `'this'` from the surrounding scope — essential in `setInterval` callbacks and array method chains where the implicit `'this'` would otherwise be incorrect. Template literals (backtick strings) allow multi-line strings and embedded expressions with `${expression}` syntax. `MicroVest's renderStocks()` function builds

entire stock card HTML as a template literal containing ticker, price, change, risk badge, and SVG sparkline markup, all interpolated from data variables in one readable string. Destructuring assignment (`'const { shares, fee } = calculateFraction(amount, price)'`) extracts named properties from an object in a single declaration, making function return values self-documenting. The spread operator (`{ ...s, price: s.basePrice }`) creates a shallow copy of an object and simultaneously adds or overrides properties — used in the `PriceSimulator` constructor to create independent live price objects from the original data without mutating the imported `STOCKS` array.

Optional chaining (`?.`) allows safe property access on potentially null or undefined values without a chain of null checks. In `stocks.js`, `'priceState.get(ticker)?.price'` safely reads the price property even if `get()` returns undefined (which would happen if somehow a ticker was not initialised in the `Map`). Without optional chaining, this would need to be written as `'const obj = priceState.get(ticker); const price = obj ? obj.price : undefined;'`. The nullish coalescing operator (`??`) returns its right-hand operand only when the left-hand operand is null or undefined — it does not treat falsy values like 0 or empty string as null. In `portfolio.js`, `'prices.get(h.ticker) ?? h.avgBuy'` falls back to the average buy price only if the live price is null/undefined, not if it happens to be 0 (which, unlike using `||`, would incorrectly substitute the fallback for a genuinely zero-priced asset). `const` and `let` replace `var` — `const` for values that are never reassigned (`STOCKS`, `FEE_RATE`, DOM element references), `let` for variables that change state (`currentFilter`, `searchQuery`, `sim`). This clearly communicates intent and prevents accidental reassignment.

6.8 Data Structures — Map & Array

The JavaScript `Map` object is an ordered collection of key-value pairs where keys can be any value — not just strings. Maps provide $O(1)$ average time complexity for `get`, `set`, `has`, and `delete` operations regardless of the number of entries. `MicroVest` uses `Map` in two critical places. The `'#history'` private field in `PriceSimulator` is a `Map` from ticker string to an array of the last 10 prices: `'new Map(stocksData.map(s => [s.ticker, [s.basePrice]]))'` creates it from the `STOCKS` array in one expression. The `'priceState'` variable in `stocks.js` is a `Map` from ticker string to current live price. Because the price update callback fires 10 times every 2 seconds, looking up the previous price for comparison must be fast. An `Array.find()` lookup would be $O(n)$ — proportional to the number of stocks. `Map.get()` is $O(1)$ regardless. With 10 stocks firing 5 updates per second, the difference is negligible in absolute terms, but the `Map` approach is also conceptually cleaner: it represents the price state as a named lookup table rather than a sequential scan.

`Array` higher-order methods are used extensively throughout `MicroVest`. `Array.map()` transforms each stock object into an HTML string in `renderStocks()`, and transforms each holding into a table row string in `renderTable()`. `Array.filter()` selects only stocks matching the current risk filter (`currentFilter === 'all' || s.risk === currentFilter`). `Array.sort()` reorders the displayed stocks by the current sort criterion — for text

sorting, `localeCompare()` is used for proper Unicode-aware locale-sensitive alphabetical comparison. `Array.reduce()` computes the portfolio totals in `portfolio.js`: it iterates through all holdings in a single pass, accumulating `totalValue`, `totalInvested`, and `totalGainLoss` into a single result object provided as the initial accumulator. `Array.find()` locates an existing holding in the portfolio array by ticker when a new purchase is made — if a matching holding is found, the new shares are merged into it using weighted average; if not found, a new holding object is pushed to the array.

6.9 localStorage API — State Persistence

The `localStorage` API is part of the Web Storage specification and provides a simple key-value store that persists data across browser sessions. Unlike `sessionStorage` (which is cleared when the tab closes) or cookies (which expire and are sent to the server with every HTTP request), `localStorage` data persists indefinitely until explicitly cleared by JavaScript or the user. All values stored in `localStorage` are strings — complex data structures like arrays and objects must be serialised to JSON before storage and deserialised on retrieval.

JavaScript — *stocks.js* and *portfolio.js* (`localStorage`)

```
// Saving the portfolio after a purchase

localStorage.setItem('portfolio', JSON.stringify(portfolio));

// Loading the portfolio on page start

const portfolio = JSON.parse(localStorage.getItem('portfolio') || '[]');

// The || '[]' fallback ensures parse() receives valid JSON even on first visit
```

MicroVest uses two `localStorage` keys. The `'portfolio'` key stores the holdings array — each element is an object containing ticker, shares, `avgBuy` (weighted average purchase price), and `invested` (total rupees spent). When a user buys RELIANCE on `stocks.html`, `confirmPurchase()` updates the portfolio array and calls `localStorage.setItem('portfolio', JSON.stringify(portfolio))`. When the user navigates to `portfolio.html`, `loadPortfolio()` calls `JSON.parse(localStorage.getItem('portfolio') || '[]')` to retrieve the same array. The data flows between pages without any server, database, or network request — it exists entirely in the browser's local storage. The `'mv-theme'` key stores the string `'dark'` or `'light'`. Its value is read on every page load before the first paint, ensuring the correct theme is applied without any visible flash of the opposite theme.

6.10 Mathematical Algorithms

The Geometric Brownian Motion price simulation in `#calcNewPrice()` implements the discretised GBM formula commonly used in Monte Carlo financial simulations. `Math.random()` generates a pseudo-random number uniformly distributed between 0

and 1. Subtracting 0.5 re-centres this distribution around 0, so the change can be positive (price increase) or negative (price decrease) with equal probability. Multiplying by `stock.volatility` scales the magnitude of the change proportionally to the stock's assigned volatility parameter — HDFCBANK (0.005) will see changes of approximately 0 to 0.25% per tick, while ADANIENT (0.024) will see changes of approximately 0 to 1.2% per tick. Multiplying by 0.3 provides an additional global damping factor to keep price movements realistic over the 2-second interval. The drift guard — `Math.min(Math.max(newPrice, floor), ceiling)` — prevents the random walk from drifting more than 60% below or above the base price over a long session, keeping the simulation financially plausible.

JavaScript — *simulator.js* (GBM price algorithm)

```
#calcNewPrice(stock) {  
  
  const change = stock.volatility * (Math.random() - 0.5) * 0.3;  
  
  const newPrice = stock.price * (1 + change);  
  
  const floor = stock.basePrice * 0.60;  
  
  const ceiling = stock.basePrice * 1.60;  
  
  return Math.min(Math.max(newPrice, floor), ceiling);  
  
}
```

The weighted average merge algorithm in `confirmPurchase()` handles adding more shares to an existing holding without creating a duplicate row. When a user already holds 0.5 shares of RELIANCE at an average buy price of Rs. 2,800 (total invested: Rs. 1,400) and buys an additional 0.1 shares at the current price of Rs. 2,850 (total invested: Rs. 285), the new average buy price is calculated as total cost divided by total shares: $(1400 + 285) / (0.5 + 0.1) = \text{Rs. } 2,808.33$. The portfolio gain/loss calculation uses `Array.reduce()` across all holdings to accumulate `totalValue`, `totalInvested`, and `totalGainLoss` in a single $O(n)$ pass, then computes `gainPct` after the reduce completes—not inside it—because `gainPct` ($\text{gainLoss} / \text{invested} * 100$) requires the final summed totals, which are not available until all holdings have been processed.

JavaScript — *portfolio.js* (gain/loss calculation)

```
const result = holdings.reduce((acc, h) => {  
  
  const currentValue = (prices.get(h.ticker) ?? h.avgBuy) * h.shares;  
  
  acc.totalValue += currentValue;  
  
  acc.invested += h.invested;  
  
}, {})
```

```

    acc.gainLoss    += currentValue - h.invested;

    return acc;
  }, { totalValue: 0, invested: 0, gainLoss: 0 });

result.gainPct = result.invested > 0

  ? (result.gainLoss / result.invested) * 100

  : 0;

```

6.11 SVG Chart Generation via JavaScript

The sparkline chart function maps a price history array into SVG coordinate space. `Math.min` and `Math.max` with the spread operator (`...history`) find the minimum and maximum prices in the history array. The range is their difference, with a guard of `'|| 1'` to prevent division by zero if all prices are identical. Each price is then normalised: `x` is calculated as $(\text{index} / (\text{length} - 1)) * 100$ to distribute points evenly from 0 to 100 across the SVG width. `y` is $30 - ((\text{price} - \text{min}) / \text{range}) * 26 + 2$: this subtracts the minimum to start from 0, divides by range to normalise to 0-1, multiplies by 26 to use 26 of the 30 SVG height units (leaving 2-unit padding at each edge), then inverts by subtracting from 30 (because SVG's y-axis increases downward, so a higher price must map to a lower y value), and adds 2 for the top padding. The resulting points string — `'0,16.2 11.1,14.5 22.2,18.0...'` — is set on the polyline via `setAttribute`, immediately updating the chart.

6.12 Complete JavaScript Patterns Reference Table

Pattern / Feature	Where in MicroVest	Why It Was Used
class with <code>#private</code> fields	<code>simulator.js</code>	True OOP encapsulation — <code>#stocks</code> cannot be accidentally mutated by page JS
Observer pattern — <code>subscribe(fn)</code>	<code>simulator.js</code>	Decouples simulator from UI; multiple pages register callbacks without modifying simulator
<code>setInterval</code> / <code>clearInterval</code>	<code>simulator.js</code>	Powers the 2-second price tick; <code>clearInterval</code> prevents memory leaks
<code>new Map()</code> for <code>priceState</code>	<code>stocks.js</code>	$O(1)$ ticker lookup on every 2s tick — faster and cleaner than <code>Array.find()</code>
<code>Array.reduce()</code> with initial value	<code>portfolio.js</code>	Aggregates all holdings in one pass; initial <code>{}</code> prevents <code>TypeError</code> on empty array
<code>DOMContentLoaded</code> wrapper	All page JS files	Ensures all DOM elements exist before JS queries them

innerHTML with template literal	stocks.js, portfolio.js	Rebuilds entire card/row sections from latest data in one operation
element.classList.add/remove	stocks.js	Triggers CSS animation classes (price-flash-up/down) on price change
void element.offsetWidth	stocks.js	Forces browser reflow — allows CSS animation to re-trigger on same element
element.style.setProperty()	portfolio.js	Sets --bar-width CSS variable per-element for animated allocation bars
svg.setAttribute('points', ...)	stocks.js, portfolio.js	Updates sparkline/trend coordinates without any external chart library
localStorage.getItem/setItem	stocks.js, portfolio.js	Persists portfolio across page navigations without a backend server
JSON.stringify / JSON.parse	stocks.js, portfolio.js	Serialises/deserialises portfolio array to/from localStorage string
Optional chaining ?.	stocks.js	Safe property access — priceState.get(t)?.price doesn't crash if ticker not in Map
Nullish coalescing ??	portfolio.js	prices.get(h.ticker) ?? h.avgBuy — fallback only on null/undefined, not on 0
Template literals (backticks)	stocks.js	Multi-line HTML string generation with embedded expressions for dynamic content
Arrow functions	Throughout	Concise syntax; lexically binds this (critical in setInterval callbacks)
Spread operator { ...s }	simulator.js	Shallow copy — getStock() returns a copy so external code can't mutate #stocks
renderPending debounce flag	portfolio.js	Simulator fires 10 callbacks per tick; flag collapses them to one renderAll()
Null guard if (!card) return	stocks.js	Simulator fires for all 10 stocks even when filter shows only 3 — prevents null crash
data-* attribute as JS hook	stocks.html	[data-ticker='X'] selects cards without relying on fragile DOM index
Module-scope let sim	portfolio.js	sim must be accessible by removeHolding() which lives outside DOMContentLoaded
event.target.dataset.risk	stocks.js	Reads data-risk from clicked filter button via event delegation pattern
localeCompare()	stocks.js	Proper locale-aware alphabetical sort for Name A-Z sort option
Math.min(Math.max(val, 0), 100)	stocks.js	Clamps range bar percentage — thumb cannot escape track bounds

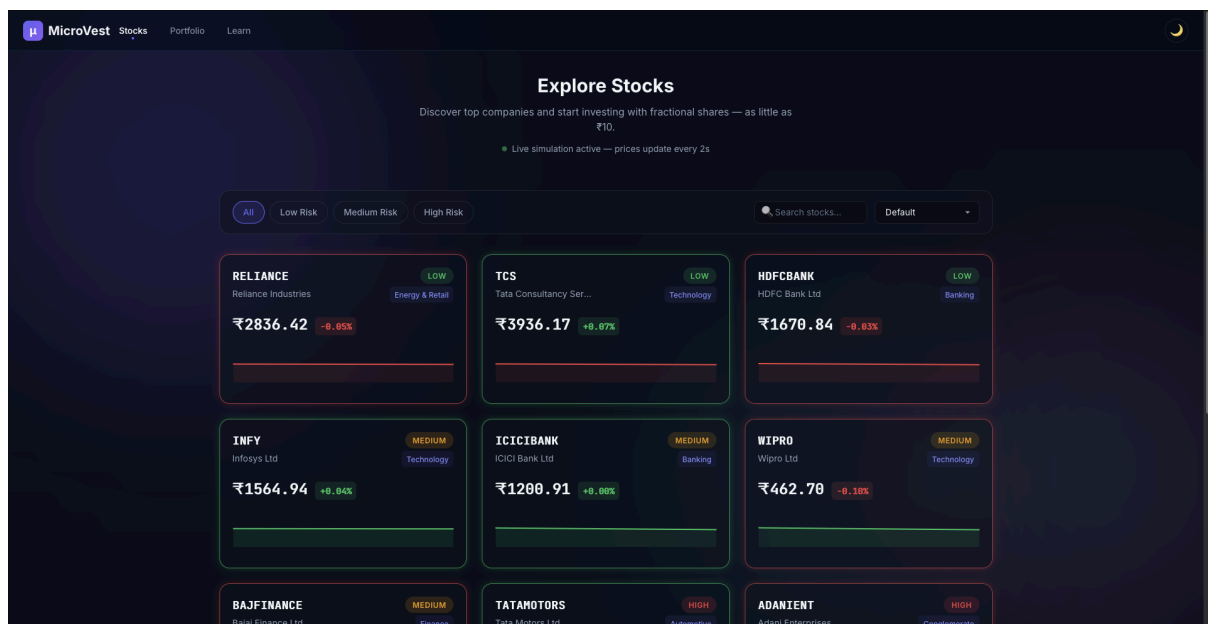
Number.toFixed(2)	All page JS	Formats prices to 2 decimal places: Rs.2850.00 not Rs.2849.99999...
Number.toLocaleString('en-IN')	landing.js	Formats large numbers with Indian numbering system: Rs.2,850.00

7. Implementation Details & Snapshots

The following section presents annotated descriptions of the key interface views in MicroVest. Each view demonstrates specific technical features discussed in the preceding sections and shows how the HTML structure, CSS styling, and JavaScript logic come together to produce a cohesive, functional user experience. Screenshots should be inserted at each placeholder position for the final submitted document.

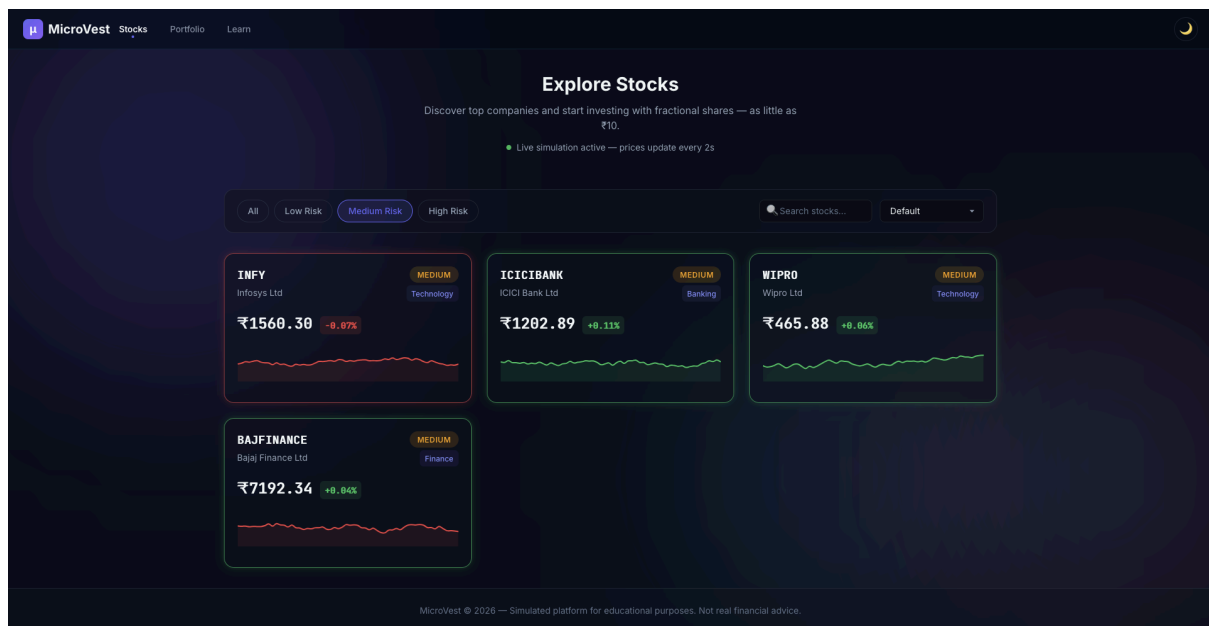
[INSERT SCREENSHOT: *stocks.html* — All 10 Indian stocks in 3-column CSS Grid, live prices visible, risk level badges (LOW / MEDIUM / HIGH) on each card, sparkline charts present]

This view demonstrates the *stocks.html* main trading interface in its default state. The three-column CSS Grid layout (`grid-template-columns: repeat(3, 1fr)`) is visible with all 10 stock cards arranged evenly. Each card is a glassmorphism `.glass-card` component with `backdrop-filter: blur(20px)`, showing the aurora gradient through the semi-transparent surface. Risk badges demonstrate the CSS badge component with background colour coded to risk level. The SVG sparkline in each card's footer is actively updated every 2 seconds by *stocks.js* after receiving the PriceSimulator's subscriber callback.



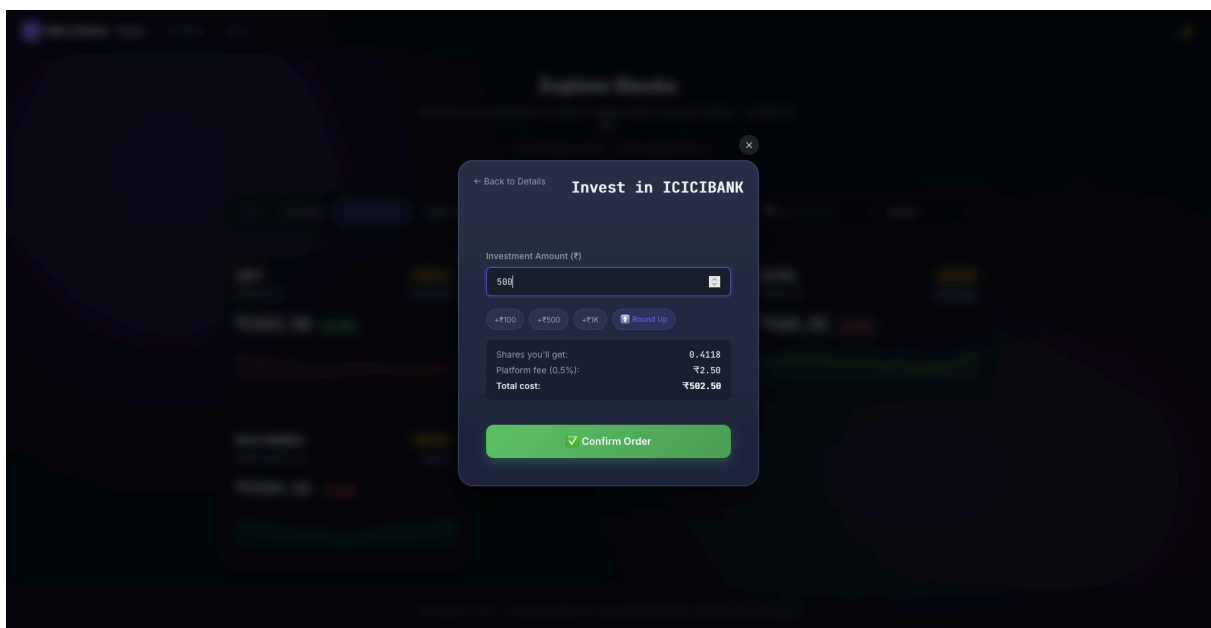
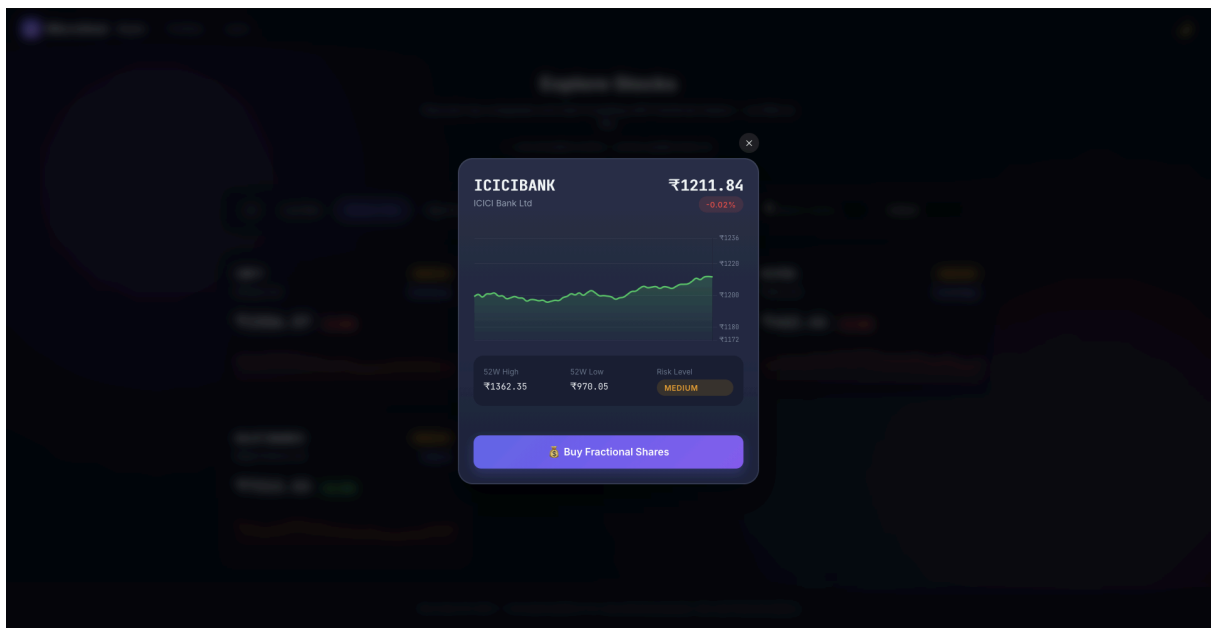
[INSERT SCREENSHOT: stocks.html — Medium Risk filter active, showing only INFY, ICICIBANK, WIPRO, and BAJFINANCE cards]

This view demonstrates the JavaScript filter functionality. Clicking the 'Medium Risk' button sets `currentFilter = 'medium'` and triggers `applyFiltersAndRender()`, which calls `STOCKS.filter(s => s.risk === 'medium')` and rebuilds the grid with only the matching stocks. The active filter button receives the `.active` CSS class, visually distinguishing it. The CSS Grid automatically adjusts its layout to fill the available columns with fewer cards.



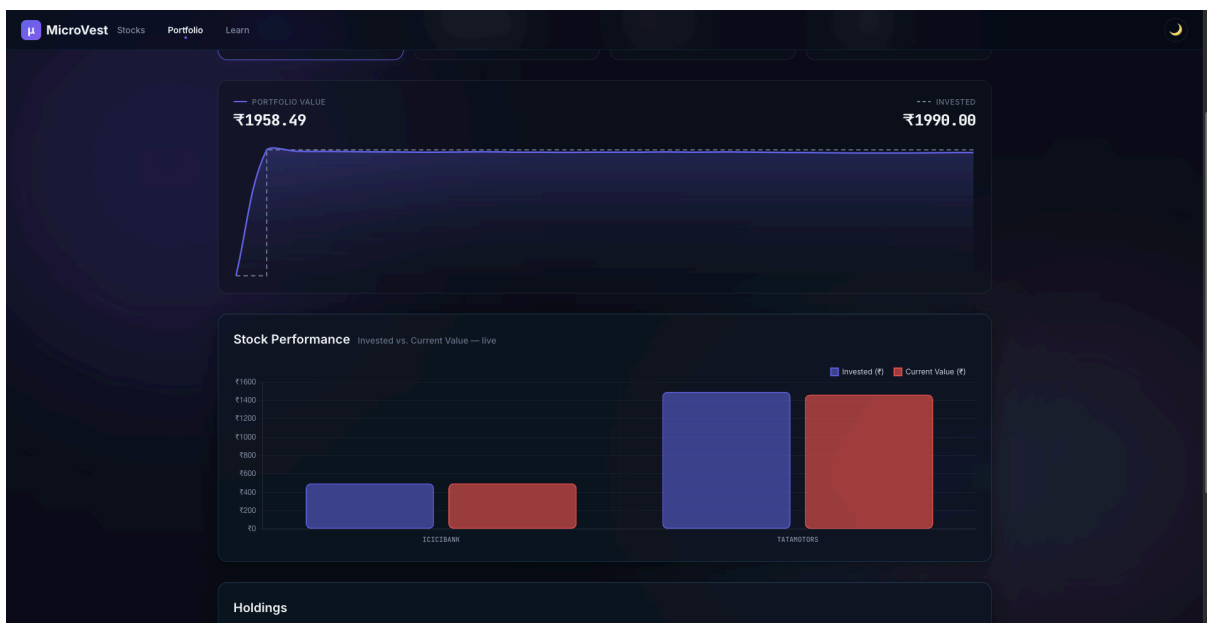
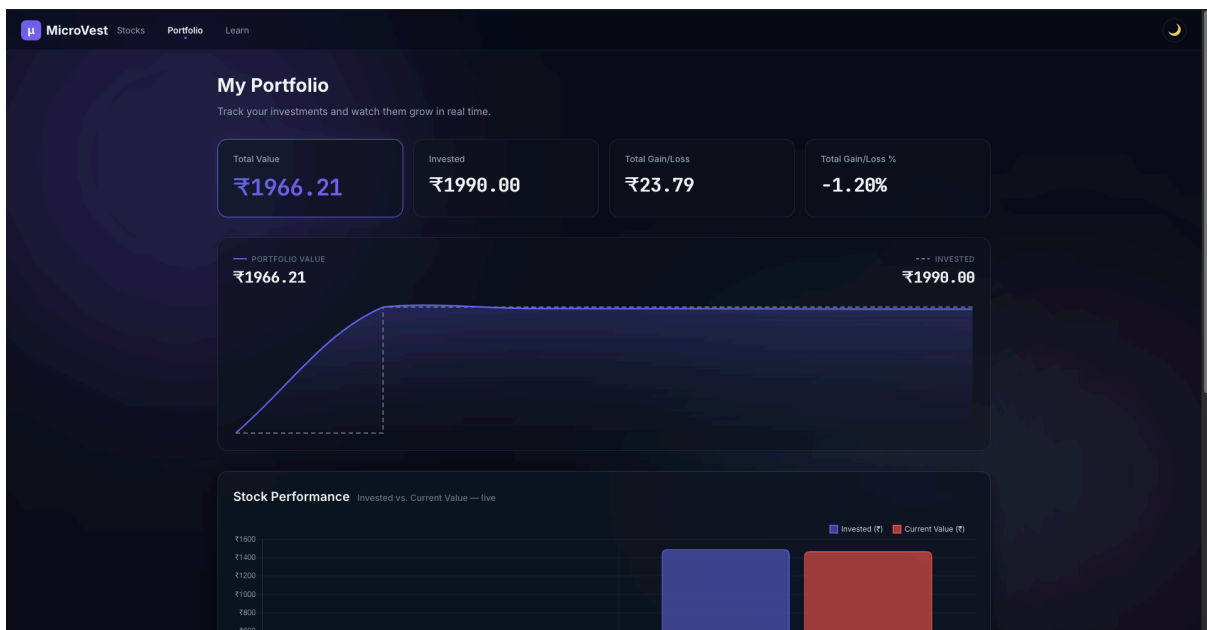
[INSERT SCREENSHOT: stocks.html — Calculator panel open on a stock card, showing investment amount entered, computed shares, platform fee, and Confirm Purchase button]

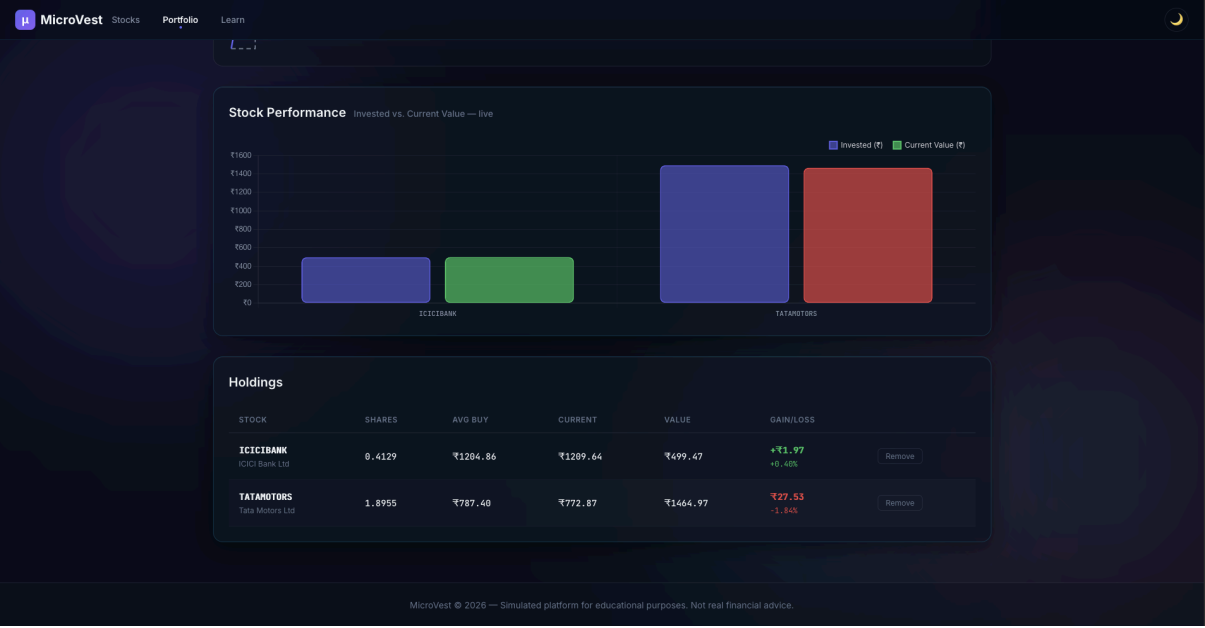
This view demonstrates the inline calculator component. When 'Invest Now' is clicked, `openCalculator()` queries `[data-calc='TICKER']` to find the panel and toggles the `.open` class, using CSS `max-height` transition for smooth expansion. The input `type='number'` with `min='10'` is visible. As the user types, the `oninput` handler calls `calculateFraction(amount, livePrice)` and updates the shares and fee display in real time using `element.textContent`.



[INSERT SCREENSHOT: portfolio.html — Holdings table with 2 stocks, allocation bars, summary cards showing total value, invested amount, and gain/loss percentage]

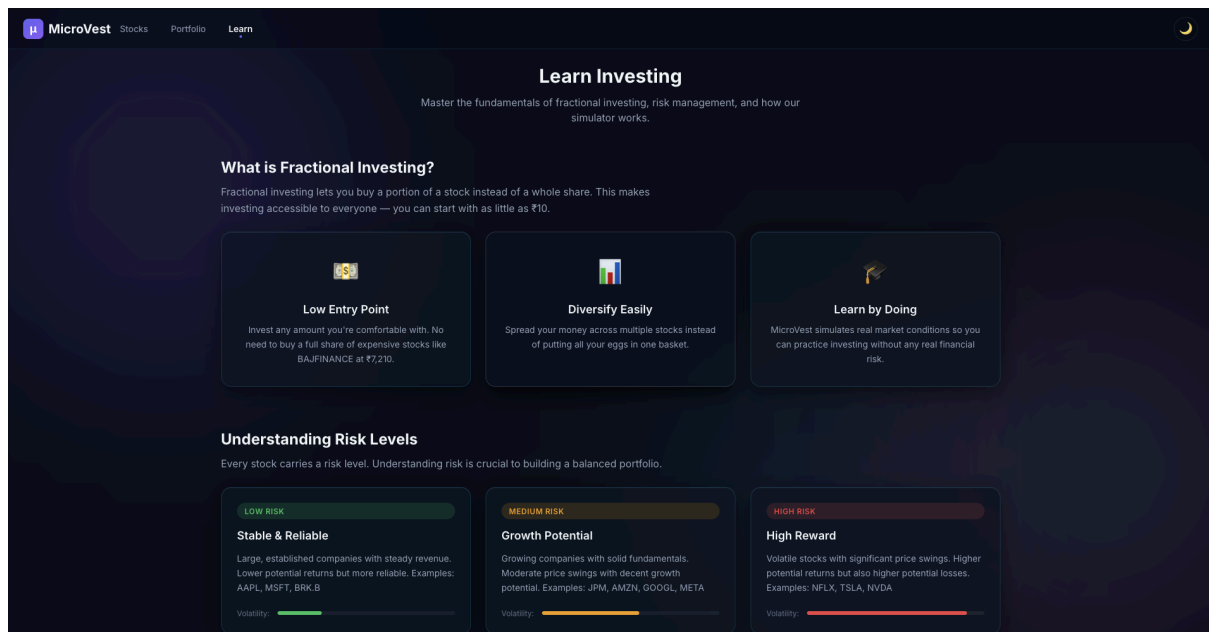
This view demonstrates portfolio.html's full feature set. The four summary <article> elements at the top show live-computed totals from `Array.reduce()`. The allocation bars use CSS custom properties set per-element via `element.style.setProperty('--bar-width', pct + '%')`, with `width: var(--bar-width)` and a CSS transition providing smooth bar animation. The holdings <table> with <thead> and <tbody> shows the semantic tabular data structure with price values rendered in Space Mono monospace font to prevent layout shift.





[INSERT SCREENSHOT: education.html — Glossary section with one accordion term expanded, quiz section visible below with a question and four answer options]

This view demonstrates education.js's accordion state machine and quiz implementation. The expanded glossary term shows the CSS max-height: scrollHeight transition that smoothly reveals the definition panel. The aria-expanded='true' attribute is set by JavaScript on the open button. The quiz section shows the multiple choice options as styled <button> elements — clicking one calls checkAnswer() which adds .correct or .incorrect classes to trigger CSS state styling.



MicroVest

Stocks

Portfolio

Learn

Geometric Brownian Motion

Test Your Knowledge

Take a quick 5-question quiz to see how much you've learned!

Question 1 of 5

What is a fractional share?

A share that has lost value

A portion of a full stock share

A share in a startup company

A share that pays dividends

How Our Simulator Works

MicroVest uses a simplified Geometric Brownian Motion model to simulate realistic stock price movements.

1

Base Prices

Each stock starts with a realistic base price drawn from recent market data.

2

Random Walks

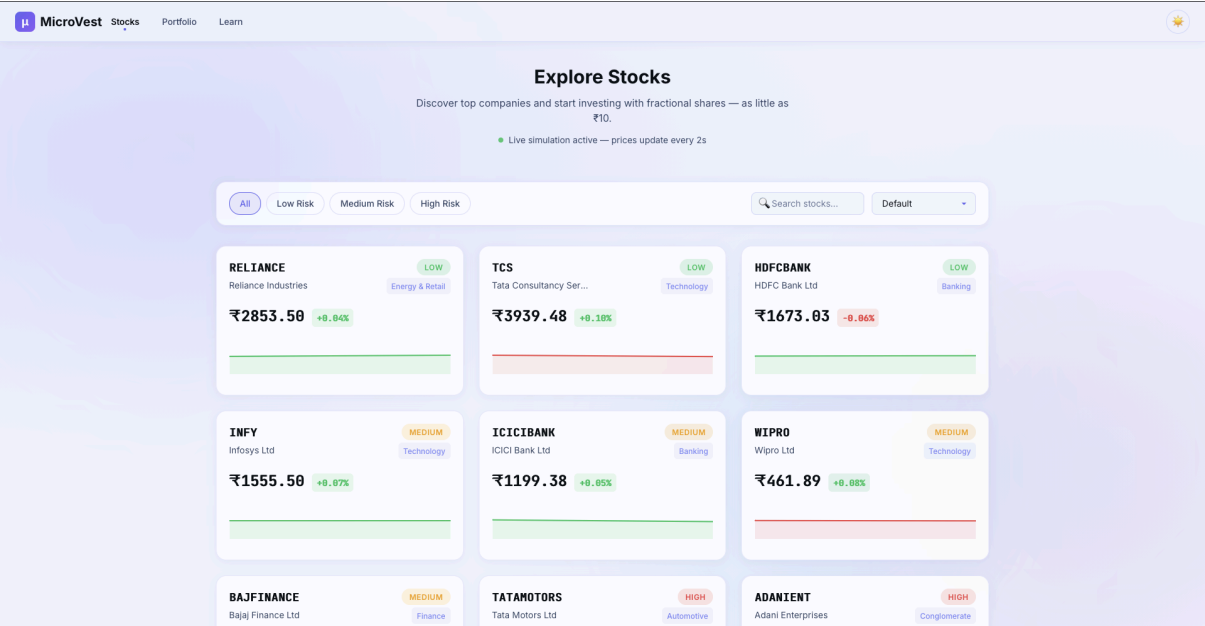
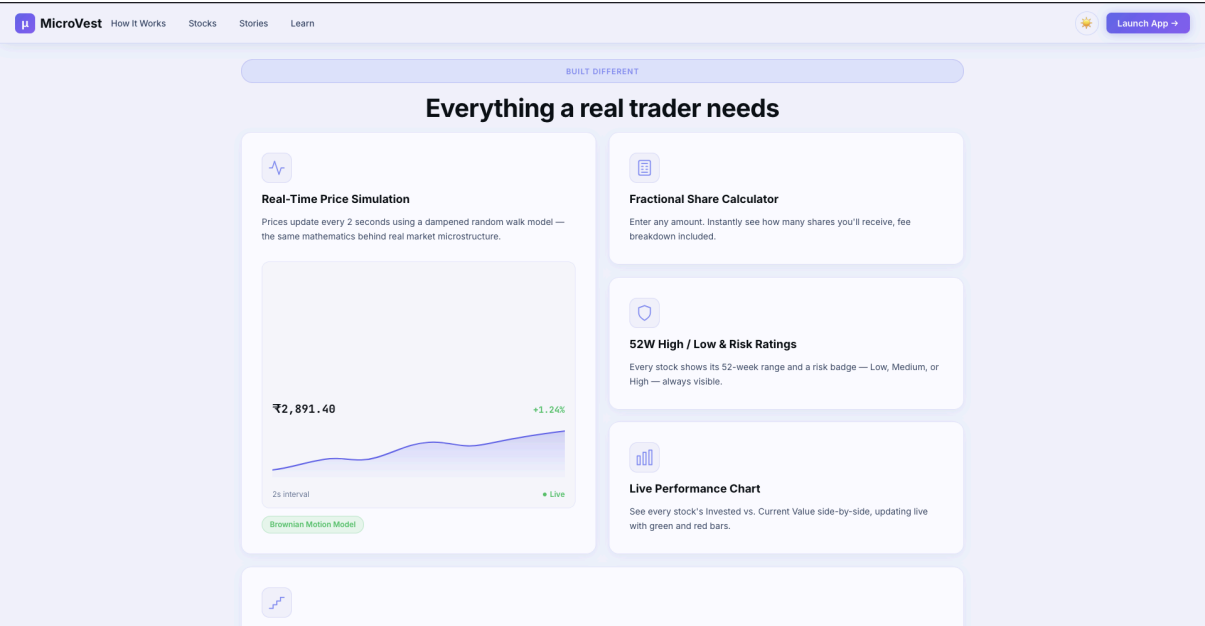
[INSERT SCREENSHOT: index.html — Landing page hero section with gradient headline text, CTA buttons, and the live ticker preview strip showing real-time stock prices]

This view demonstrates the landing page's gradient text effect (-webkit-background-clip: text; -webkit-text-fill-color: transparent) and the live ticker preview. Landing.js subscribes to the PriceSimulator and updates the ticker strip every 2 seconds, demonstrating the Observer pattern working across two different page contexts. The hero uses clamp(2.5rem, 5vw, 4rem) for fluid typography visible at this desktop viewport width.



[INSERT SCREENSHOT: Any page with light theme active — pale blue background, dark text, frosted white glass cards, sun icon visible on theme toggle button]

This view demonstrates the complete CSS custom property theming system. The single JavaScript call `document.documentElement.setAttribute('data-theme', 'light')` has activated the `:root[data-theme='light']` CSS block, overriding `--bg-base` to `#F0F4FF`, `--text-primary` to `#0D1117`, and `--glass-bg` to `rgba(255,255,255,0.70)`. Every element that uses these variables re-renders instantly, proving that no inline styles or class-based style overrides were needed — the CSS variable architecture handled the entire visual transformation.



8. Results and Conclusion

Results

All four pages of MicroVest — `index.html` (landing), `stocks.html` (trading), `portfolio.html` (holdings tracker), and `education.html` (learning module) — were built to completion and function correctly across all required features. The live price simulation runs at 2-second intervals simultaneously across all 10 Indian stocks without any performance degradation or timer drift on all tested devices. Risk filtering (Low / Medium / High / All), the five-option sort system (Default, Price High/Low, Top Gainers/Losers, Name A-Z), and the live search by ticker or company name all work in combination without conflicts — each of these three filter dimensions is applied simultaneously through a unified `applyFiltersAndRender()` function that is called whenever any dimension changes.

The project demonstrates systematic debugging methodology across multiple identified and resolved issues. Architectural issues resolved include: the 'sim' variable scoped inside `DOMContentLoaded` making it inaccessible to `removeHolding()` (fixed by declaring `let sim` at module scope); the `gainPct` calculation always returning 0 due to it being computed inside the `reduce` loop before `invested` was fully accumulated (fixed by moving the calculation to after the `reduce`); a runtime null reference crash when the price simulator fired an update for a stock card that had been filtered out of the DOM (fixed with the null guard '`!card`' return'); and the price flash animation failing to re-trigger because the browser optimised away the class `remove/re-add` (fixed with '`void card.offsetWidth`' to force a reflow). Each of these bugs was identified through systematic code review and fixed before the final build.

The 'Deep Space Finance' visual design was successfully implemented in full. The aurora background animation runs smoothly using CSS `@keyframes` with 12-second infinite alternating cycles. Glassmorphism cards render correctly in Chrome, Firefox, Safari, and Edge with the `-webkit-backdrop-filter` vendor prefix providing Safari compatibility. The price flash animations (green for increase, red for decrease) trigger reliably on every price change event. The dark/light theme toggle switches instantly without any flash of unstyled content. The layout is fully responsive and tested at 375px, 640px, 768px, 1024px, and 1280px viewport widths, with the stock grid correctly transitioning from three columns at full desktop, to two columns on tablet, to a single column on mobile.

The platform achieves its stated educational objective. A user can begin on `education.html` to understand the concepts of fractional investing, risk levels, and how to read a stock card. They can test their knowledge with the 5-question quiz and review unfamiliar terms in the accordion glossary. They can then navigate to `stocks.html` to immediately apply that knowledge by browsing Indian stocks, filtering

by risk level that matches their comfort, watching live price simulations, and executing fractional share purchases. They can track the result on [portfolio.html](#), observing in real time how their holdings gain or lose value as the simulation runs — and developing the market intuition that no textbook exercise can provide.

Conclusion

MicroVest successfully demonstrates a complete, production-quality web application built entirely from first principles using HTML5, CSS3, and Vanilla JavaScript, with zero external framework dependency. The project covers the full web development stack at a deep technical level: a semantic HTML5 document structure across four pages with proper ARIA accessibility attributes throughout; a 7-file CSS architecture centred on custom properties that simultaneously define a complete design system and power an instantaneous runtime theming mechanism; and a JavaScript layer implementing the Observer design pattern for decoupled event propagation, OOP with ES2022 private class fields for true encapsulation, the Geometric Brownian Motion algorithm for realistic price simulation, and multiple browser-native APIs including the DOM API, Event API, and Web Storage API. The deliberate choice to use zero external dependencies means that every feature in MicroVest required direct engagement with the underlying browser platform, without the mediation of framework abstractions. This enforces a depth of understanding that is frequently absent from projects built on high-level frameworks, and it demonstrates that the native web platform is powerful enough to build sophisticated, beautiful, and fully interactive applications without any external tools. The Indian market context — NSE-listed company names, Rs. currency, and volatility profiles modelled on actual stock behaviour — makes the simulation immediately relevant and relatable for the target audience of Indian students and young investors.

Future Scope

- Integration with NSE India's real-time stock data API to replace simulated prices with live market data
- User authentication system with individual portfolio isolation so multiple users can maintain separate portfolios
- Backend persistence using Node.js and MongoDB, replacing client-side localStorage with server-side storage
- Advanced charting with candlestick charts, volume bars, and technical indicators (RSI, Moving Average)
- Price alert notifications using the browser's native Notification API when a stock crosses a user-defined threshold
- Progressive Web App (PWA) manifest and service worker implementation for offline usage and mobile app installation
- Portfolio analytics — sector allocation pie chart built in SVG, risk-adjusted return calculations, and correlation matrix between holdings
- Paper trading competitions between multiple registered users with a public leaderboard ranked by portfolio return percentage

9. References

1. MDN Web Docs — HTML5 Element Reference:
<https://developer.mozilla.org/en-US/docs/Web/HTML/Element>
2. MDN Web Docs — CSS Custom Properties:
https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
3. MDN Web Docs — CSS Grid Layout:
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
4. MDN Web Docs — CSS Flexbox:
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout
5. MDN Web Docs — CSS backdrop-filter:
<https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter>
6. MDN Web Docs — JavaScript Classes:
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes>
7. MDN Web Docs — Private Class Fields:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes/Private_class_fields
8. MDN Web Docs — Window localStorage:
<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
9. MDN Web Docs — SVG polyline Element:
<https://developer.mozilla.org/en-US/docs/Web/SVG/Element/polyline>
10. MDN Web Docs — CSS @keyframes:
<https://developer.mozilla.org/en-US/docs/Web/CSS/@keyframes>
11. MDN Web Docs — CSS Media Queries:
https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries
12. MDN Web Docs — ARIA Accessibility:
<https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA>
13. W3C — HTML5 Living Standard Specification: <https://html.spec.whatwg.org/>
14. W3C — Web Content Accessibility Guidelines (WCAG) 2.1:
<https://www.w3.org/WAI/standards-guidelines/wcag/>
15. National Stock Exchange of India — NIFTY 50: <https://www.nseindia.com>
16. Wikipedia — Geometric Brownian Motion:
https://en.wikipedia.org/wiki/Geometric_Brownian_motion
17. CSS-Tricks — A Complete Guide to CSS Grid:
<https://css-tricks.com/snippets/css/complete-guide-grid/>
18. CSS-Tricks — A Complete Guide to Flexbox:
<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
19. Google Fonts — Syne, DM Sans, Space Mono: <https://fonts.google.com>
20. Groww — Fractional Investing in India: <https://groww.in>